EPHEC
ÉCOLE SUPÉRIEURE
ENSEIGNEMENT POUR ADULTES

Année académique **2026-2027**

Programmation orientée objet — Java

Introduction et annexes

Le module d'introduction, le guide IntelliJ et les mémentos

À lire avant le premier module

Ce document a été élaboré avec l'aide de Claude, un assistant d'intelligence artificielle développé par Anthropic. L'enseignant en a conçu le contenu et la progression, l'a relu, corrigé et validé, et en assume la responsabilité.

EPHEC — Bachelier en informatique, niveau 2

Table des matières

INI — Introduction au cours 4
INI §1 — Le cours 5
INI §2 — Ma façon de travailler 7
INI §3 — Les documents de référence 8
En résumé 9
Annexe A — Guide IntelliJ IDEA 10
À quoi sert ce guide 10
Sommaire 10
A — Avant de commencer 12
B — Ouvrir le projet du cours 12
C — L'écran 15
D — Écrire du code 18
E — Exécuter 21
F — Transformer du code sans le casser 25
G — Naviguer 26
H — Raccourcis et réglages 27
Annexe — Erreurs et solutions 31
Annexe B — Mémento technique 36
À quoi sert ce mémento 36
Sommaire 36
La plateforme Java 37
Les interfaces graphiques 39
De votre code au programme qui tourne 41
Ce qui se passe pendant l'exécution 42
Maven 44
Recevoir les fichiers du cours 46
Git, en deux mots 47
Annexe C — Mémento Clean Code 49
À quoi sert ce mémento 49
Les règles en un coup d'œil 49
Nommer 51
Écrire une méthode 55
Commenter 59
Objets et encapsulation 60
Traiter les erreurs 61
Écrire des tests 64
Structurer une classe 66
La règle du boy-scout 69

Pour aller plus loin 69
Index 70

Module INI

INI — Introduction au cours

Ce que couvre le cours, comment il s'organise, et comment travailler

Cette introduction présente le cours : ce qu'il couvre et ce que vous devrez savoir faire au terme de l'année, comment il s'organise et comment il est évalué, puis la façon dont s'articulent les différents supports, et les documents de référence qui vous accompagneront toute l'année.

INI §1 — Le cours

Qui suis-je ?

Avant d'entrer dans le cours, quelques mots sur celui qui le donne.

Mon parcours professionnel a commencé au helpdesk d'ING, qui s'appelait encore la BBL à l'époque. Je suis ensuite devenu testeur, puis développeur : c'est à ce moment-là que j'ai suivi, à l'EPHEC, le même parcours que vous. Je suis devenu ensuite analyste fonctionnel, puis Scrum Master, et j'évolue aujourd'hui vers un rôle de coach, d'équipe ou d'organisation.

Ce qui m'a d'abord attiré vers l'informatique, c'est sa capacité à rendre visibles certains objets mathématiques, comme l'ensemble de Mandelbrot ou le jeu de la vie de Conway.

J'aime les années 80, qui ont bercé mon adolescence : Simple Minds, Depeche Mode, le film *Blade Runner*. J'aime aussi le café.

Enfin, je suis attentif, sans doute davantage depuis que j'ai trois enfants, aux problématiques systémiques dans lesquelles notre monde est plongé : les ressources, l'énergie, la biodiversité. C'est ce qui m'a poussé à m'inscrire à une formation en développement durable.

Contenu du cours

Ce cours approfondit et met en pratique ce que vous avez vu en C#. Il aborde des concepts plus avancés de la programmation orientée objet, comme l'héritage, les types génériques et les expressions lambda, quelques classes de la bibliothèque standard de Java, et quelques outils, dont `JUnit` pour les tests et `JavaFX` pour les interfaces graphiques.

Chaque notion est expliquée, puis pratiquée au travers d'exercices. Le travail personnel entre les séances est indispensable : c'est lui qui permet d'assimiler ce qui a été vu au cours. Si vous rencontrez une difficulté, signalez-la au plus vite plutôt que de la laisser s'installer.

Les modules du cours

Le cours est découpé en modules. Chacun porte un code de trois lettres, que vous retrouverez partout : sur les slides, dans le nom des exercices, dans les packages Java et sur Moodle. Le code désigne le sujet, pas la séance : un module peut occuper moins d'une séance, ou en déborder.

Code	Sujet
`POO`	Programmation orientée objet : classes et objets
`TMP`	Temps : dates et durées
`TAB`	Tableaux
`ENU`	Énumérations et records
`ABS`	Héritage, classes abstraites
`ITF`	Interfaces
`PKG`	Packages et visibilité
`COL`	Collections
`EXC`	Exceptions
`JUN`	Tests unitaires (`JUnit`)
`GEN`	Types génériques
`LAM`	Classes anonymes et expressions lambda
`FX`	Interfaces graphiques (`JavaFX`)
`FIC`	Fichiers
`REC`	Exercices récapitulatifs

L'ordre est indicatif. Il peut évoluer en cours d'année ; Moodle indique, au fil des séances, les modules abordés.

Les acquis d'apprentissage

Les acquis d'apprentissage, repris du dossier pédagogique du cours, décrivent ce que vous devez être capable de faire au terme du cours. Ils se regroupent en deux familles.

La première porte sur les objets : l'étudiant sera capable de concevoir, d'installer et d'utiliser des objets appropriés à la solution. Concrètement, il pourra :

- définir et implémenter des classes et des objets Java simples ;
- compléter des classes Java implémentées par un tiers ;
- utiliser des classes et des objets Java existants, et interagir avec eux ;
- dériver de nouvelles classes par héritage d'une classe Java existante ;
- utiliser les classes du JDK pour accéder à des fichiers, et pour réaliser une application disposant d'une interface graphique simple (`JavaFX`) ;
- démontrer, au travers d'un exercice simple, sa compréhension du concept et du fonctionnement des expressions lambda.

La seconde porte sur les tests : l'étudiant sera capable de concevoir et de mettre en œuvre une procédure de test partiel et intégré. Il pourra :

- implémenter des classes de test simples (`JUnit`), à partir d'une classe implémentée par un tiers ou de spécifications techniques ;
- implémenter des classes et des objets Java simples, et les classes de test correspondantes ;
- repérer et corriger des problèmes simples dans des classes écrites par un tiers, en exécutant une batterie de tests `JUnit`.

En d'autres termes, vous devez pouvoir concevoir des classes et les utiliser, employer l'héritage à bon escient, écrire et exécuter des tests qui vérifient qu'une classe fonctionne, et utiliser les classes du JDK vues au cours : fichiers, collections, exceptions et événements. Les deux premiers points prolongent ce que vous connaissez déjà en C#.

L'organisation pratique

Le cours compte quatorze séances, du 19 janvier au 18 mai 2027. Chaque séance commence à 18 h 00 et se termine au plus tard à 21 h 35, avec une pause de quinze minutes, et alterne théorie, démonstration et exercices.

Vous travaillerez avec Java et l'environnement de développement IntelliJ IDEA. Maven gère la structure des projets et les bibliothèques, `JUnit` sert aux tests et `JavaFX` aux interfaces graphiques. Les versions à utiliser sont annoncées sur Moodle : travaillez avec celles-là toute l'année, ce sont celles de l'examen.

> **Attention** - Présence aux séances
>
> Message officiel de la direction de l'EPHEC : « En cas de dépassement des 40 % d'absences injustifiées dans une unité d'enseignement, l'étudiant se verra automatiquement désinscrit du cours et non-admis à l'examen. »

L'examen

L'examen a lieu le 8 juin 2027. Il se déroule à cahier ouvert : il ne demande pas de par cœur, mais la capacité à écrire du code qui fonctionne. Il se passe sur machine et dure deux heures trente.

Il compte cinq ou six questions, dont la forme reste stable d'une session à l'autre :

- écrire une classe simple, qui lève une exception ;
- écrire un type générique ;
- écrire et exécuter une classe de tests pour une classe donnée ;

- écrire une hiérarchie d'interfaces et de classes ;
- compléter une application graphique pour qu'elle fonctionne ;
- et peut-être une sixième question, dont l'existence et le sujet vous seront confirmés en cours d'année.

Chaque question correspond à un ou plusieurs modules. Les exercices récapitulatifs de fin d'année en reprennent la structure.

INI §2 — Ma façon de travailler

Le déroulé d'un module

Un module suit toujours le même chemin :

Étape	Ce qu'elle apporte
Les slides	les notions, présentées en séance
La démonstration	les notions mises en pratique devant vous, en direct dans IntelliJ
Les exercices	votre pratique, à partir du fascicule d'exercices du module
GameHub	le fil rouge, un projet plus grand que vous faites évoluer à chaque séance

Les slides ne contiennent pas les énoncés : une slide de renvoi indique le code des exercices à faire, par exemple `POO.1`, et l'énoncé complet se trouve dans le fascicule d'exercices du module.

Les fascicules d'exercices

Chaque module a son fascicule d'exercices, distribué en PDF ; le recueil des exercices les rassemble en un seul document. Les exercices y sont numérotés par sujet : `TAB.3` est le troisième exercice sur les tableaux, quelle que soit la séance où on le fait. Le même code donne le nom du package dans le projet IntelliJ : `TAB.3` se trouve dans `be.ephec.java.tab.e03`.

Tous les énoncés suivent la même structure : l'objectif, le contexte, l'énoncé, les spécifications à respecter, la manière de vérifier votre code, et les questions à se poser. Cette dernière rubrique est la plus importante : elle signale ce que l'exercice cherche à vous faire découvrir.

Le projet ExercicesJava

Les exercices sont livrés dans un projet Maven, ExercicesJava, à ouvrir dans IntelliJ. À chaque séance, une archive zip ajoute les exercices du jour ; les solutions sont distribuées après le cours. Le guide IntelliJ et le mémento technique expliquent comment ouvrir le projet.

Les classes sont déjà écrites, avec leurs méthodes. Le corps de chaque méthode se contente de lever une `UnsupportedOperationException`, qui signifie « pas encore écrit » :

```
public int computeAge(int annee) {
    throw new UnsupportedOperationException(
            "computeAge n'est pas encore implementee");
}
```

Le projet compile donc dès le départ : c'est à vous de remplacer ces lignes par le vrai code.

À l'examen, en revanche, certaines questions vous demanderont de créer vous-même une classe qui n'existe pas encore : le fichier, la déclaration de la classe et ses méthodes (voir `IDE.8` du guide IntelliJ). Profitez des exercices pour vous y entraîner, en écrivant parfois une classe de toutes pièces plutôt qu'en complétant le squelette fourni.

Les tests fournis

La plupart des exercices sont accompagnés d'une classe de test `JUnit`. Vous la lancez vous-même, aussi souvent que vous le voulez : c'est votre moyen de savoir si votre réponse est correcte, sans attendre la séance suivante. Un test rouge signale que la réponse n'est pas encore correcte ; quand tous les tests sont verts, elle l'est.

Un test vert ne dit pas que votre code est bien écrit, seulement qu'il fait ce qui est demandé. Et les tests ne vérifient jamais un point que l'énoncé laisse à votre choix.

Vous n'avez pas besoin de connaître `JUnit` pour lancer ces tests : le mémento Exercices explique comment faire, et le module `JUN` vous apprendra à écrire les vôtres.

Le fil rouge GameHub

GameHub est une plateforme de jeux rudimentaire, construite petit à petit au fil de l'année : chaque module y ajoute une pièce, qui s'appuie sur ce qui a été construit précédemment. Ses énoncés sont numérotés `G.1`, `G.2`, et ainsi de suite, dans l'ordre imposé par leurs dépendances. GameHub vit dans un projet séparé des exercices.

Là où un exercice isole une notion, GameHub vous oblige à la faire cohabiter avec tout ce que vous avez déjà écrit. C'est la situation que vous retrouverez à l'examen, et surtout dans tout projet réel.

Les tests de module et leurs badges

Chaque module a, sur Moodle, un test : huit questions tirées au hasard parmi quinze. Après chaque réponse, vous voyez si elle est juste et, sinon, la partie du support à revoir. Le test est réussi à 6 sur 8, et vous pouvez le refaire autant de fois que vous le voulez : les questions changent d'une tentative à l'autre.

Chaque test réussi vous donne un badge. Quatre badges d'étape marquent les grandes parties du cours, et un dernier, « Java, de bout en bout », la réussite des quatorze tests. Un tout premier badge, « Pied à l'étrier », vous accueille dès que vous avez téléchargé le projet de départ.

Ces badges ne comptent pas dans la note. Ce sont des repères, pour voir où vous en êtes et ce qu'il vous reste à revoir : prenez-les comme un encouragement, à votre rythme.

Ce support de cours

Les slides accompagnent l'explication orale. Ce support reprend chaque notion en prose, avec le code formaté, module par module. C'est lui qu'il faut relire entre les séances.

INI §3 — Les documents de référence

Quatre documents

À côté des fascicules et de ce support, quatre documents de référence vous accompagnent toute l'année. On ne les lit pas d'une traite, à part le mémento Exercices : on les ouvre quand un énoncé ou une slide y renvoie. Vous les trouverez en annexe de ce support, et sur Moodle, dans la section Documents.

Document	Préfixe	À quoi il sert
Mémento Exercices		lancer les tests, lire un résultat
Guide IntelliJ	`IDE`, `ERR`	se servir de l'outil, comprendre une erreur
Mémento technique	`TEC`	comprendre ce qui se passe sous le capot
Mémento Clean Code	`CC`	écrire du code lisible

Le **mémento Exercices** rassemble ce qui vaut pour tous les fascicules, et qu'il serait inutile de répéter dans chacun. Lisez-le une fois en début d'année. Il explique la numérotation des exercices, montre comment lancer les classes de test fournies et lire leur résultat, et pose une règle :

> **À retenir** - gardez toujours un code qui compile
>
> Tout le dossier `src/main/java` se compile d'un seul bloc : un exercice laissé à moitié écrit empêche de lancer tous les autres, y compris ceux des séances précédentes. Avant de passer à autre chose, vérifiez que votre code compile. Sinon, deux issues : le corriger, ou mettre le morceau en chantier entre commentaires avec `Ctrl+/` (`⌘/` sur Mac). C'est aussi ce que l'examen demande : on y rend du code qui compile.

Le **guide IntelliJ** couvre ce qu'il faut savoir d'IntelliJ pour ce cours, captures d'écran à l'appui : l'installation, la création et l'ouverture d'un projet, l'exécution, le débogage, le paramétrage. Ses entrées `ERR` partent du message affiché et expliquent sa cause la plus probable : c'est le premier réflexe à avoir quand quelque chose ne fonctionne pas.

Le **mémento technique** explique ce qu'IntelliJ fait à votre place : ce que sont la JVM, le JRE et le JDK, comment un fichier `.java` devient un programme, comment Maven récupère les bibliothèques, comment se lance une application `JavaFX`. Vous pouvez écrire vos premiers programmes sans le lire, mais il répond à la plupart des « pourquoi ça marche ? ».

Le **mémento Clean Code** rassemble 37 règles simples, en sept sections (noms, méthodes, commentaires, objets, erreurs, tests, classes), pour écrire du code qu'un autre développeur, ou vous-même dans six mois, comprendra sans effort. Un programme qui fonctionne n'est pas forcément un programme lisible. Ces règles s'inspirent du livre *Clean Code* de Robert C. Martin. Elles ne font pas l'objet d'un module : elles arrivent par petites touches, au fil des exercices, quand elles deviennent utiles. Elles s'appliquent aussi à l'examen : un code qui fonctionne mais ne les respecte pas peut y perdre des points.

Comment lire un renvoi

Les fascicules, les slides et ce support renvoient aux mémentos par des codes :

Renvoi	Où chercher
`IDE.14`	la section 14 du guide IntelliJ
`CC-MET.8`	la règle 8 de la section « méthodes » du mémento Clean Code
`TEC-EXE.4`	la section 4 de la partie « exécution » du mémento technique
`POO §6`	la section 6 du module `POO`, dans ce support

Un renvoi vous dit où chercher, au moment où vous en avez besoin. Par exemple, l'exercice `POO.5` vous demande de lire une Javadoc : il vous renvoie à `IDE.14`, qui montre comment l'afficher sans quitter IntelliJ. Le premier renvoi arrive dès le module `POO` : prenez l'habitude de le suivre.

Quelques liens utiles

La documentation officielle de Java, et notamment la Javadoc qui décrit chaque classe de la bibliothèque standard, se trouve sur le site d'Oracle : `docs.oracle.com/en/java/javase/`. Choisissez la version de Java annoncée sur Moodle : les classes évoluent d'une version à l'autre.

Le site de Jean-Michel Doudoux, `jmdoudoux.fr/java/dej/`, propose en français un tour très complet du langage et de sa bibliothèque.

En résumé

- Un module, c'est un sujet, avec son code de trois lettres.
- Chaque module suit le même chemin : slides, démonstration, exercices du fascicule, GameHub.
- Des tests vous permettent de vérifier vous-même vos réponses.
- Les mémentos s'ouvrent quand un renvoi vous y envoie.

Le module suivant, `POO`, entre dans le vif du sujet : il commence par ce qui change quand on passe de C# à Java.

Annexe A

Annexe A — Guide IntelliJ IDEA

Installer, ouvrir, écrire, exécuter, dépanner

À quoi sert ce guide

IntelliJ IDEA est l'outil dans lequel vous passerez toutes vos heures de Java cette année. Ce guide n'en fait pas le tour — personne n'en fait le tour — mais il couvre ce dont vous aurez besoin, dans l'ordre où vous en aurez besoin.

Les deux premières sections servent le premier soir : installer, et ouvrir le projet du cours. Les suivantes servent toute l'année et se consultent au coup par coup. L'annexe rassemble les messages d'erreur les plus fréquents, avec leur cause et leur remède.

Chaque section porte un code — `IDE.17` pour le lancement des tests, par exemple — et les énoncés d'exercices y renvoient au moment utile. Les codes sont cliquables dans ce document.

Deux conventions

Les noms de menus sont donnés en anglais, tels qu'ils apparaissent à l'écran dans l'installation par défaut : `File → Project Structure`. Les explications, elles, restent en français.

Les raccourcis sont donnés dans les deux systèmes, séparés par une barre oblique : `Ctrl+Alt+L` sous Windows et Linux, `⌥⌘L` sous macOS. Les symboles Mac se lisent ⌘ commande, ⌥ option, ⇧ majuscule, ^ contrôle.

Les captures d'écran

Elles ont été prises sous macOS avec un thème sombre. Sous Windows, les fenêtres ont une autre apparence mais le même contenu, et les menus portent les mêmes noms. Ne vous laissez pas troubler par les couleurs.

Sommaire

Avant de commencer	
IDE.1	Quelle édition d'IntelliJ
IDE.2	Installer IntelliJ
IDE.3	Le JDK ne vient pas avec IntelliJ

Ouvrir le projet du cours	
IDE.4	Ouvrir par le pom.xml, jamais par le dossier
IDE.5	Installer un JDK depuis IntelliJ
IDE.6	Vérifier le SDK du projet

L'écran	
IDE.7	Les quatre zones
IDE.8	La fenêtre Project, créer une classe
IDE.9	La fenêtre Maven
IDE.10	Ouvrir et fermer les fenêtres d'outils

Écrire du code	
IDE.11	Complétion et import automatique
IDE.12	Générer accesseurs et constructeurs
IDE.13	Les modèles : psvm, sout, fori
IDE.14	Lire la Javadoc sans quitter l'éditeur
IDE.15	Les avertissements et leurs corrections

Exécuter	
IDE.16	Lancer une classe
IDE.17	Lancer les tests et lire le rapport
IDE.18	Passer des arguments au programme
IDE.19	Le débogueur

Transformer du code sans le casser	
IDE.20	Renommer
IDE.21	Déplacer une classe
IDE.22	Extraire une méthode

Naviguer	
IDE.23	Aller à la déclaration, et revenir
IDE.24	Chercher n'importe quoi
IDE.25	La structure d'un fichier

Raccourcis et réglages	
IDE.26	Les raccourcis à connaître
IDE.27	Plugins et apparence
IDE.28	JavaFX : rien à configurer, ou presque
IDE.29	Installer Scene Builder

Annexe — Erreurs et solutions	
ERR.1	JDK isn't specified for module
ERR.2	Cannot resolve symbol
ERR.3	Tout est rouge sans raison
ERR.4	Pas de flèche verte sur une classe de test
ERR.5	Deux classes du même nom

Annexe — Erreurs et solutions	
ERR.6	Lancer malgré une erreur de compilation
ERR.7	JavaFX runtime components are missing
ERR.8	source level should be in … : 21
ERR.9	Un exercice inachevé bloque tous les autres
ERR.10	Un fichier disparaît de la fenêtre Project
ERR.11	Les modules Maven ont disparu : plus rien ne se lance

A — Avant de commencer

IDE.1 — Quelle édition d'IntelliJ

IntelliJ existe en deux éditions. L'édition **Community** est gratuite et suffit intégralement pour ce cours : elle gère Java, Maven, `JUnit` et le débogueur. L'édition **Ultimate** est payante et ajoute des outils dont nous n'aurons pas l'usage — serveurs d'applications, frameworks web, bases de données.

Prenez Community. Rien dans ce guide ni dans les exercices ne demande Ultimate.

IDE.2 — Installer IntelliJ

Téléchargez depuis `jetbrains.com/idea/download`, en veillant à prendre la version correspondant à votre machine. Sur un Mac récent, choisissez la variante `Apple Silicon` et non `Intel` : la seconde fonctionne, mais plus lentement.

L'installation ne demande aucun réglage particulier. Au premier démarrage, IntelliJ propose d'importer des paramètres existants : répondez que vous n'en avez pas.

IDE.3 — Le JDK ne vient pas avec IntelliJ

C'est la source de confusion la plus fréquente en début d'année, alors autant l'énoncer tout de suite.

IntelliJ est un *éditeur*. Il ne compile pas votre code lui-même : il confie ce travail à un JDK, qui est un logiciel distinct. Installer IntelliJ n'installe donc pas Java, et sans JDK rien ne compilera.

IntelliJ affiche bien un numéro de version de Java dans sa fenêtre `About` — c'est celui du Java qui fait tourner IntelliJ lui-même, pas celui qui compilera votre code. Les deux n'ont aucun rapport.

Deux façons d'installer un JDK. Soit vous le téléchargez chez son éditeur et vous l'installez comme n'importe quel logiciel. Soit vous le laissez faire à IntelliJ, ce qui est plus simple et se fait à l'étape suivante, une fois le projet ouvert. Nous prendrons cette seconde voie : voir IDE.5.

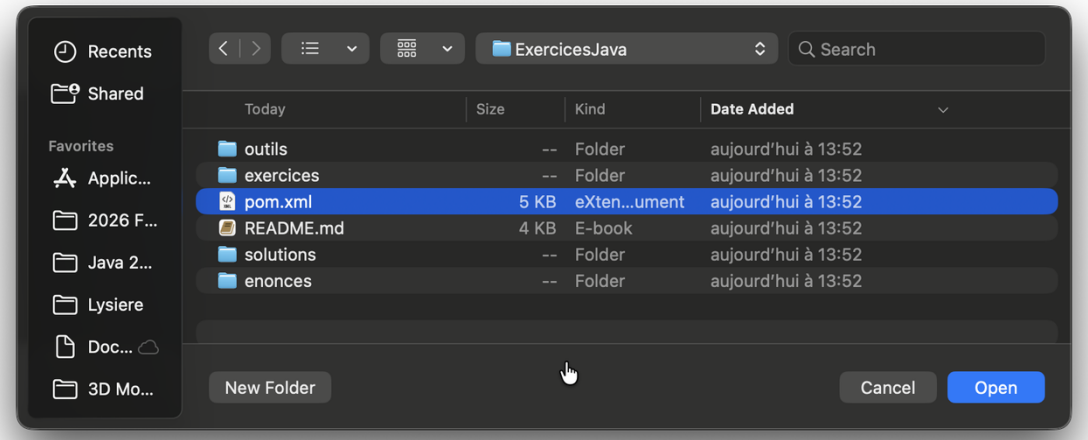
> **Quelle version, quelle distribution**
>
> Ce cours utilise **Java 21**, qui est une version à support long terme. N'installez pas une version plus récente : votre code pourrait compiler chez vous et pas à l'examen.
>
> La distribution — Amazon Corretto, Eclipse Temurin, Oracle — importe peu : ce sont des empaquetages du même Java. Corretto et Temurin sont gratuits sans condition.

B — Ouvrir le projet du cours

IDE.4 — Ouvrir par le pom.xml, jamais par le dossier

Le projet du cours est un projet **Maven** composé de trois modules Maven, `exercices`, `solutions` et `demos`. Sa description tient dans un fichier `pom.xml` placé à la racine.

`File → Open`, puis sélectionnez le fichier `pom.xml` lui-même — pas le dossier qui le contient. IntelliJ propose alors « Open as Project » : acceptez.



Sélectionner le `pom.xml` de la racine, et non le dossier.

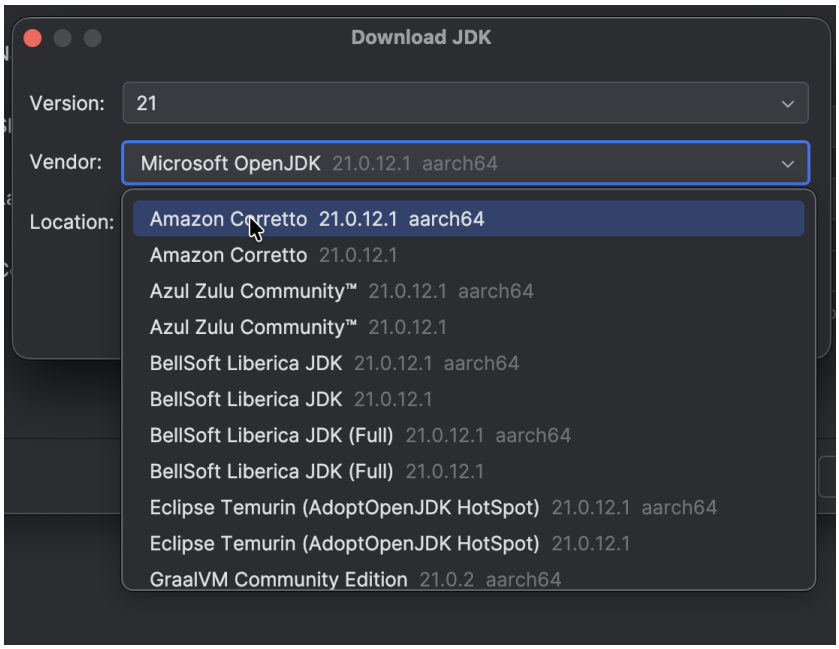
Le `pom.xml` est le fichier qui décrit le projet à Maven : ses modules, ses bibliothèques, sa version de Java. Le mémento technique le détaille en `TEC-MVN.2`, et explique à quoi sert Maven en `TEC-MVN.1`. Notre projet en compte plusieurs modules — voir `TEC-MVN.3`.

Pourquoi cette insistance. Si vous ouvrez le dossier, IntelliJ crée parfois des modules ordinaires au lieu de modules Maven. Ces modules n'ont pas de JDK, ne connaissent pas les bibliothèques déclarées dans le `pom.xml`, et rien ne compile. C'est la cause de l'erreur décrite en `ERR.1`.

Au premier chargement, IntelliJ télécharge les bibliothèques dont le projet a besoin — `JUnit`, `JavaFX`. Comptez une à deux minutes, avec une barre de progression en bas de la fenêtre. C'est un téléchargement unique : les fois suivantes, l'ouverture est immédiate.

IDE.5 — Installer un JDK depuis IntelliJ

`File → Project Structure → Project`. Dépliez la liste du champ `SDK` : elle contient les JDK qu'IntelliJ a trouvés sur votre machine, et une entrée `Download JDK…` en bas.



La liste du champ SDK, avec l'entrée Download JDK.

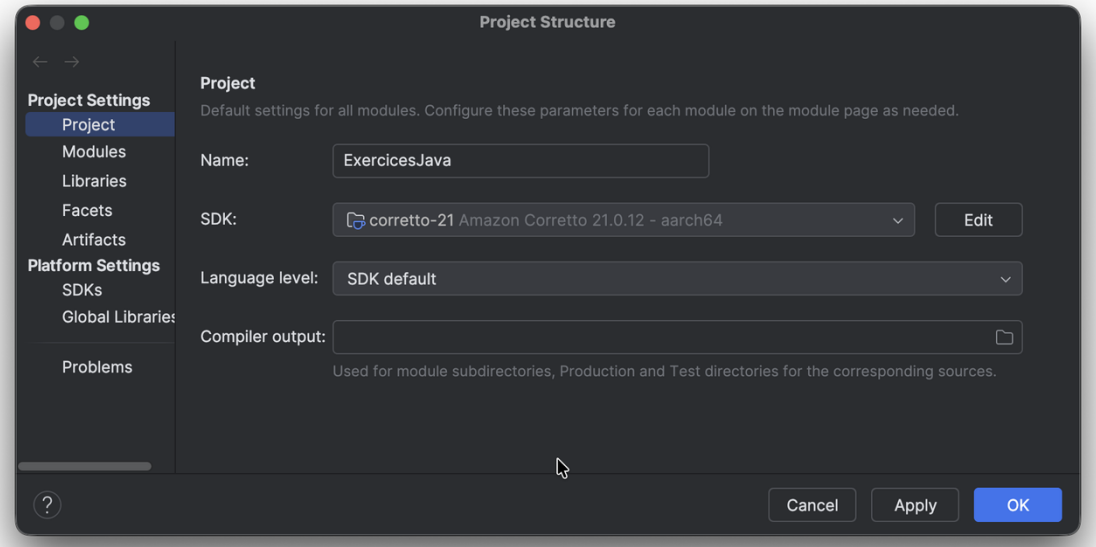
Choisissez la version 21 et une distribution Amazon Corretto ou Eclipse Temurin. IntelliJ télécharge et installe le JDK tout seul, puis le sélectionne.

Attention à l'architecture. Sur un Mac Apple Silicon, prenez la variante `aarch64` et non `x86_64`. Si la liste ne le précise pas, elle propose déjà la bonne.

IDE.6 — Vérifier le SDK du projet

Toujours dans `File → Project Structure → Project`, deux champs comptent.

- **SDK** : le JDK qui compilera votre code. Il doit nommer un JDK 21.
- **Language level** : la version du langage que vous êtes autorisé à écrire. Mettez-la sur 21 également.

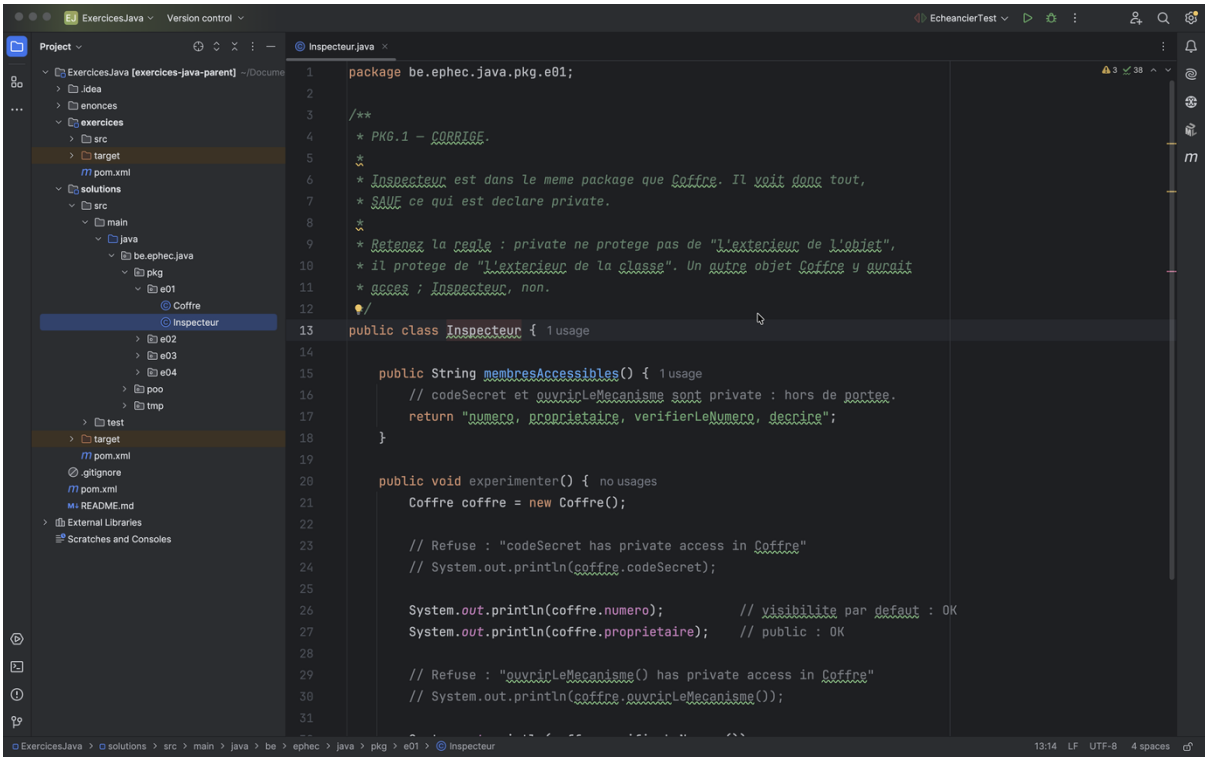


Les deux champs à vérifier : SDK et Language level.

Les modules Maven n'ont pas de JDK propre : ils héritent de celui du projet. Vous n'avez donc rien à régler module par module — et si un module affiche un JDK explicite au lieu de « Project SDK », c'est une anomalie à corriger. Voir ERR.1.

C — L'écran

IDE.7 — Les quatre zones



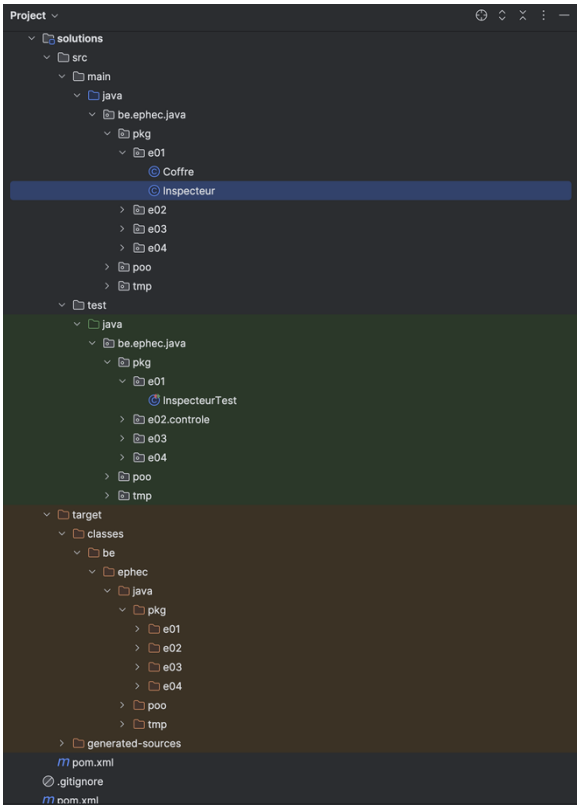
IntelliJ, projet ouvert.

L'écran se lit en quatre zones.

- **À gauche**, les fenêtres d'outils : Project le plus souvent, mais aussi Maven, Structure, Debug. Elles s'ouvrent et se ferment à volonté.
- **Au centre**, l'éditeur, avec un onglet par fichier ouvert. C'est là que vous passerez l'essentiel de votre temps.
- **En bas**, les fenêtres de résultat : Run pour l'exécution, Debug pour le débogage, Terminal. Elles apparaissent quand vous lancez quelque chose.
- **En haut à droite**, la barre de lancement : la configuration courante, la flèche verte, l'insecte du débogueur.

Tout le reste — les bandeaux de boutons sur les bords — n'est qu'un raccourci vers ces mêmes fenêtres. Ne cherchez pas à tout retenir : trois ou quatre suffisent, et les autres se découvriront d'elles-mêmes.

IDE.8 — La fenêtre Project, créer une classe



L'arborescence du projet, dépliée.

L'arborescence a l'air compliquée, mais elle obéit à une convention Maven simple et invariable.

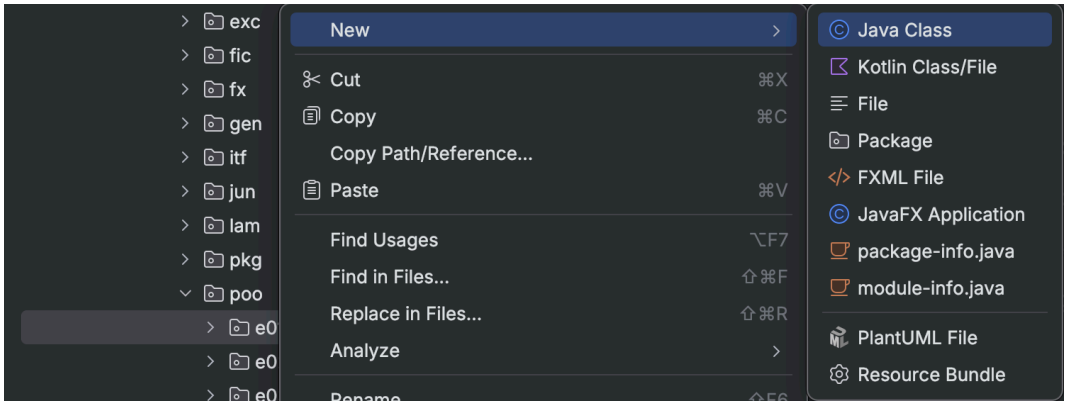
Dossier	Contenu
`src/main/java`	Votre code. Les classes que vous écrivez.
`src/test/java`	Les classes de test. Fournies, à ne pas modifier.
`target`	Le résultat de la compilation. Généré, jamais édité à la main.
`pom.xml`	La description du projet : modules, bibliothèques, version de Java.

Le dossier `target` se recrée intégralement à chaque compilation. C'est pourquoi il est ignoré par Git et absent des archives : il ne contient rien qu'on ne puisse refabriquer. Le mémento technique décrit ce qu'il contient en `TEC-CMP.4`, et ce qui distingue votre source, le bytecode et le programme en cours d'exécution en `TEC-CMP.1`.

Notez aussi que `src/main/java` et `src/test/java` contiennent les *mêmes* noms de packages. Ce n'est pas une erreur : une classe de test vit dans le même package que la classe qu'elle teste, ce qui lui donne accès à ses membres non publics.

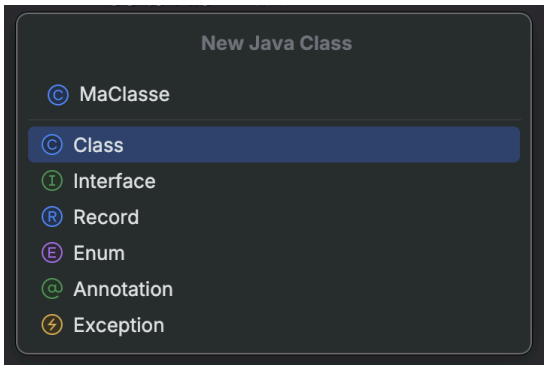
Créer une classe

Dans les exercices, les classes existent déjà. À l'examen, certaines questions vous demandent d'en créer une. Faites un clic droit sur le package qui doit l'accueillir, par exemple `be.ephec.java.poo.e01`, puis choisissez New, puis Java Class.



Clic droit sur le package, New, puis Java Class.

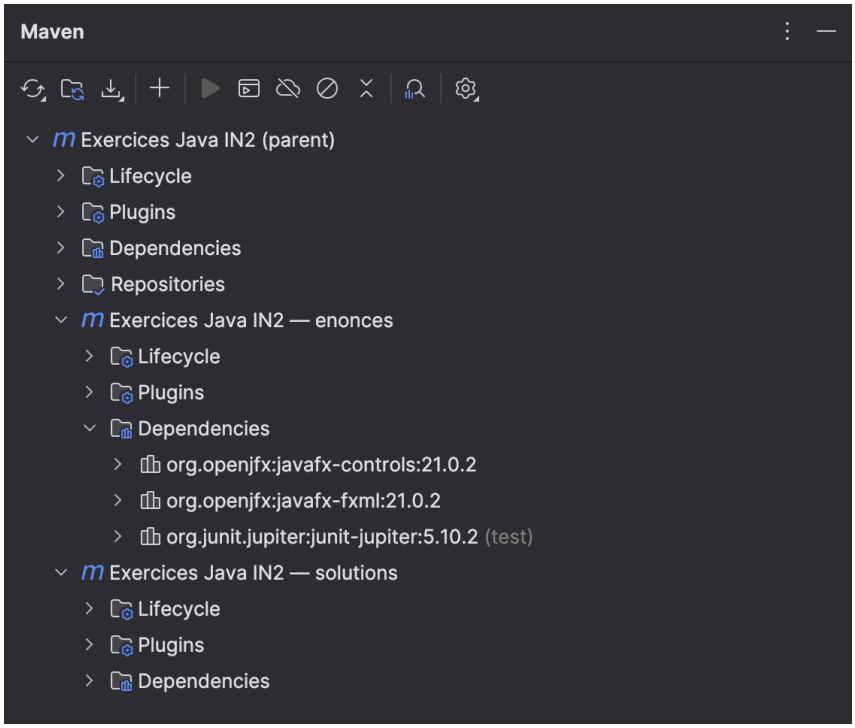
IntelliJ demande alors le nom de la classe et son genre : une classe ordinaire, une interface, un `record`, une énumération (`enum`)… Le nom suit les conventions : une majuscule initiale, sans espace ni accent, et sans l'extension `.java`, qu'IntelliJ ajoute lui-même.



Le nom de la classe, puis son genre.

Le fichier créé contient déjà la ligne `package` et la déclaration de la classe, vide : il ne reste qu'à l'écrire. Choisir le bon package compte : une classe créée ailleurs ne sera pas trouvée par la classe de test qui l'attend.

IDE.9 — La fenêtre Maven



Le projet parent et ses modules.

Cette fenêtre montre le projet tel que Maven le voit : un projet parent, et sous lui un dossier par module — `exercices`, `solutions` et `demos`. Si l'un d'eux n'y figure pas, c'est qu'IntelliJ ne l'a pas reconnu comme module Maven, et il faut relire `IDE.4`.

Le bouton de rechargement, en haut à gauche de cette fenêtre, force IntelliJ à relire les `pom.xml`. C'est le premier geste à faire quand une bibliothèque semble introuvable alors qu'elle est déclarée. Voir `ERR.2`.

Sous chaque module, quatre dossiers. Deux vous serviront, les deux autres presque jamais.

Dossier	Ce qu'il contient
`Lifecycle`	les étapes de construction : clean, compile, test, package…
`Dependencies`	les bibliothèques du module, telles que le `pom.xml` les déclare
`Plugins`	les outils que Maven ajoute, comme javafx ou surefire
`Repositories`	les serveurs où Maven va chercher ce qui lui manque

Dépliez Lifecycle : vous y trouverez les étapes du cycle de construction. Un double-clic sur l'une d'elles la lance, et Maven exécute au passage toutes celles qui la précèdent — lancer `test` compile donc d'abord. Le mémento technique détaille cet enchaînement en `TEC-MVN.4`.

Dépliez Dependencies : vous y verrez les bibliothèques réellement utilisées, avec leur version. C'est l'endroit où vérifier qu'une dépendance déclarée dans le `pom.xml` est bien arrivée.

Elles ne sont téléchargées qu'une fois : Maven les conserve ensuite sur votre machine, ce qui explique pourquoi le premier chargement d'un projet est long et les suivants immédiats. Voir `TEC-MVN.5`.

IDE.10 — Ouvrir et fermer les fenêtres d'outils

Les fenêtres d'outils portent un numéro, et le raccourci qui les ouvre reprend ce numéro. Le même raccourci les referme, ce qui rend l'écran modulable en une frappe.

Fenêtre	Windows / Linux	macOS
Project	`Alt+1`	`⌘1`
Run	`Alt+4`	`⌘4`
Debug	`Alt+5`	`⌘5`
Terminal	`Alt+F12`	`⌥F12`
Tout masquer sauf l'éditeur	`Ctrl+Maj+F12`	`⇧⌘F12`

La dernière ligne est la plus utile : elle bascule entre un écran chargé et l'éditeur seul, plein écran. Pratique quand on écrit, et pour se retrouver quand tout est ouvert.

D — Écrire du code

IDE.11 — Complétion et import automatique

IntelliJ propose des complétions au fil de la frappe : tapez les premières lettres et la liste apparaît. Entrée valide la proposition surlignée.

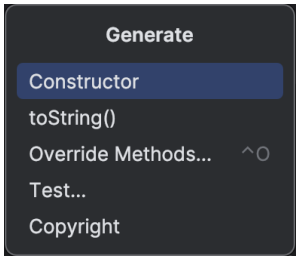
Le point important est ailleurs. Quand vous acceptez une complétion sur une classe qui n'est pas encore importée — `Scanner`, `LocalDate`, `ArrayList` — IntelliJ ajoute l'`import` en tête de fichier tout seul. Vous n'avez jamais à écrire un `import` à la main.

Si un nom de classe apparaît en rouge parce que son `import` manque, placez le curseur dessus et faites `Alt+Entrée` / `⌥⏎` : la première proposition sera de l'importer.

Une nuance qui compte. La complétion propose parfois plusieurs classes de même nom venant de bibliothèques différentes. `Date` existe en trois versions, et `KeyEvent` en deux — une pour `JavaFX`, une pour AWT. Lisez le nom complet affiché à droite de la proposition avant de valider : importer la mauvaise donne un code qui ne compile pas, avec un message qui n'aide pas.

IDE.12 — Générer accesseurs et constructeurs

Placez le curseur dans le corps d'une classe et faites `Alt+Inser` / `⌘N`. Le menu Generate propose de fabriquer ce qui se déduit mécaniquement des champs.



Le menu Generate.

- **Getter and Setter** : les accesseurs des champs que vous choisissez.
- **Constructor** : un constructeur initialisant les champs sélectionnés.
- **`Override` Methods** : les méthodes héritées que vous voulez redéfinir.

> **Attention aux doublons**
>
> Les squelettes d'exercices déclarent souvent déjà les accesseurs, avec un corps qui lève une exception. Dans ce cas, complétez ce corps — ne régénérez pas, vous obtiendriez deux méthodes du même nom et la classe ne compilerait plus.
>
> Générer ne dispense pas de réfléchir : un getter et un setter pour chaque champ annule l'encapsulation qu'on vient de mettre en place. Voir la règle `CC-OBJ.2` du mémento Clean Code.

IDE.13 — Les modèles : psvm, sout, fori

Certaines suites de caractères se déplient en code quand on les tape puis qu'on appuie sur Tab. Trois valent d'être connues dès la première séance.

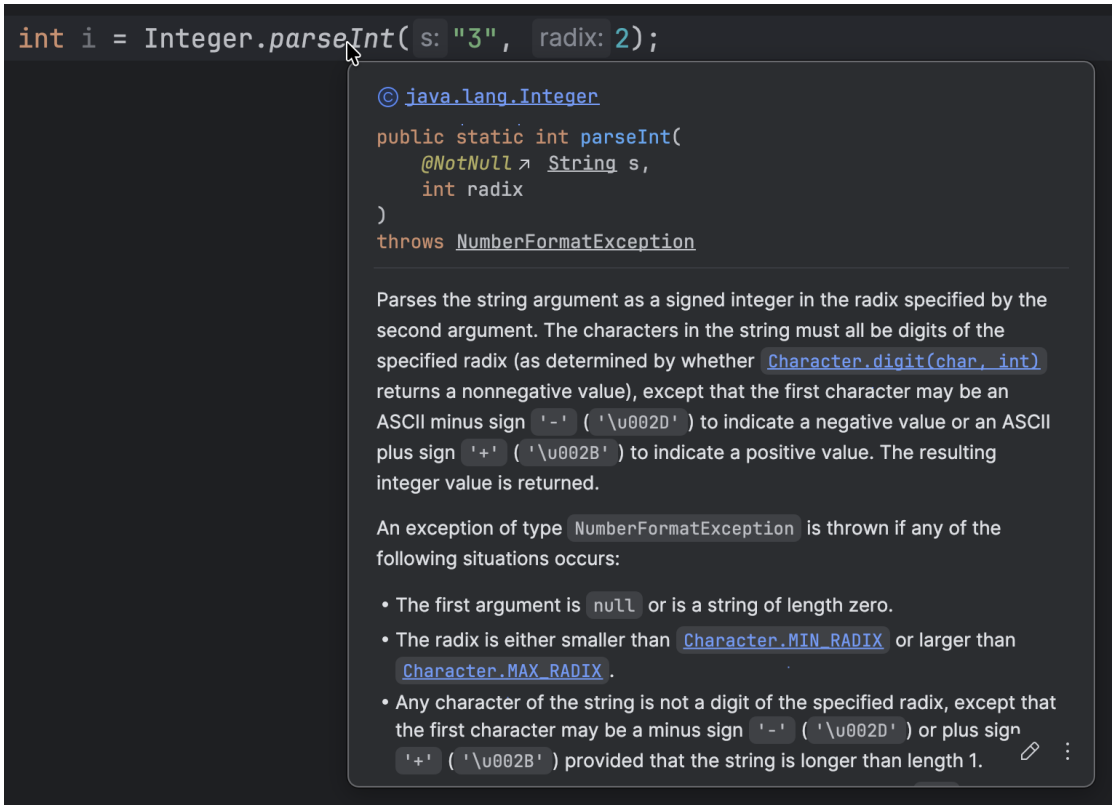
Vous tapez	Vous obtenez
`psvm`	`public static void main(String[] args) { }`
`sout`	`System.out.println();`
`fori`	`une boucle for indexée complète`
`iter`	`une boucle for-each sur la collection la plus proche`

Il en existe des dizaines d'autres. `Ctrl+J` / `⌘J` affiche la liste de ceux disponibles à l'endroit où vous êtes.

IDE.14 — Lire la Javadoc sans quitter l'éditeur

Plusieurs exercices vous demandent d'aller lire la documentation d'une méthode du JDK. Vous n'avez pas besoin d'ouvrir un navigateur : la documentation est dans IntelliJ.

Survolez le nom de la méthode avec la souris, ou placez-y le curseur et faites `Ctrl+Q` / `F1`.



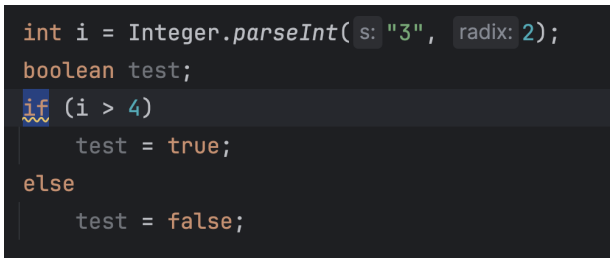
La documentation de `Integer.parseInt(String, int)`.

Cette bulle contient tout ce qu'un énoncé peut vous demander de chercher : la signature exacte, ce que la méthode retourne, et surtout la liste des cas où elle lève une exception. Sur la capture, on lit que le radix doit se situer entre `MIN_RADIX` et `MAX_RADIX` — exactement ce que demande l'exercice `POO.5`.

Les liens de la bulle sont cliquables : de proche en proche, on remonte à toute la bibliothèque standard.

IDE.15 — Les avertissements et leurs corrections

Un souligné rouge signale une erreur : le code ne compile pas. Un souligné jaune ou gris signale autre chose, et c'est celui qui nous intéresse ici — le code compile parfaitement, mais IntelliJ estime qu'on peut faire mieux.



Le code compile. Le souligné jaune signale autre chose.

Survolez la partie soulignée : le message explique ce qui est en cause.



Le diagnostic au survol.

Puis `Alt+Entrée` / ⌥⏎ ouvre la liste des corrections possibles. La première est en général la bonne, et un aperçu montre le résultat avant que vous ne validiez.



Les corrections proposées, avec l'aperçu du résultat.

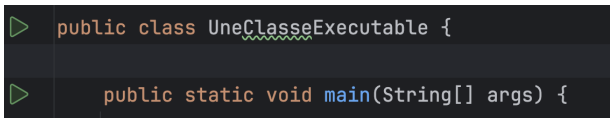
Dans cet exemple, un `if/else` qui affecte `true` dans une branche et `false` dans l'autre ne fait que recopier sa propre condition. IntelliJ propose de le remplacer par `test = i > 4;`, qui dit la même chose en une ligne.

> **Ces avertissements recoupent le mémento Clean Code**
>
> La variable jamais utilisée, la méthode qui pourrait être `static`, la condition inutilement compliquée : IntelliJ effectue gratuitement une partie du travail de relecture que le mémento vous demande de faire à la main.
>
> Prenez l'habitude de lire ces messages au lieu de les ignorer. Ils ne sont pas tous pertinents — vous apprendrez à faire le tri — mais ils sont bien plus souvent justes que le contraire.

E — Exécuter

IDE.16 — Lancer une classe

Une classe est exécutable si elle contient une méthode `main`. IntelliJ le repère et affiche une flèche verte dans la marge, à hauteur de la déclaration.



La flèche verte, dans la marge, à côté du main.

Cliquez dessus et choisissez `Run`. La fenêtre Run s'ouvre en bas et affiche la sortie du programme. Le raccourci `Maj+F10` / `^R` relance ce que vous avez lancé la dernière fois, sans repasser par la marge.

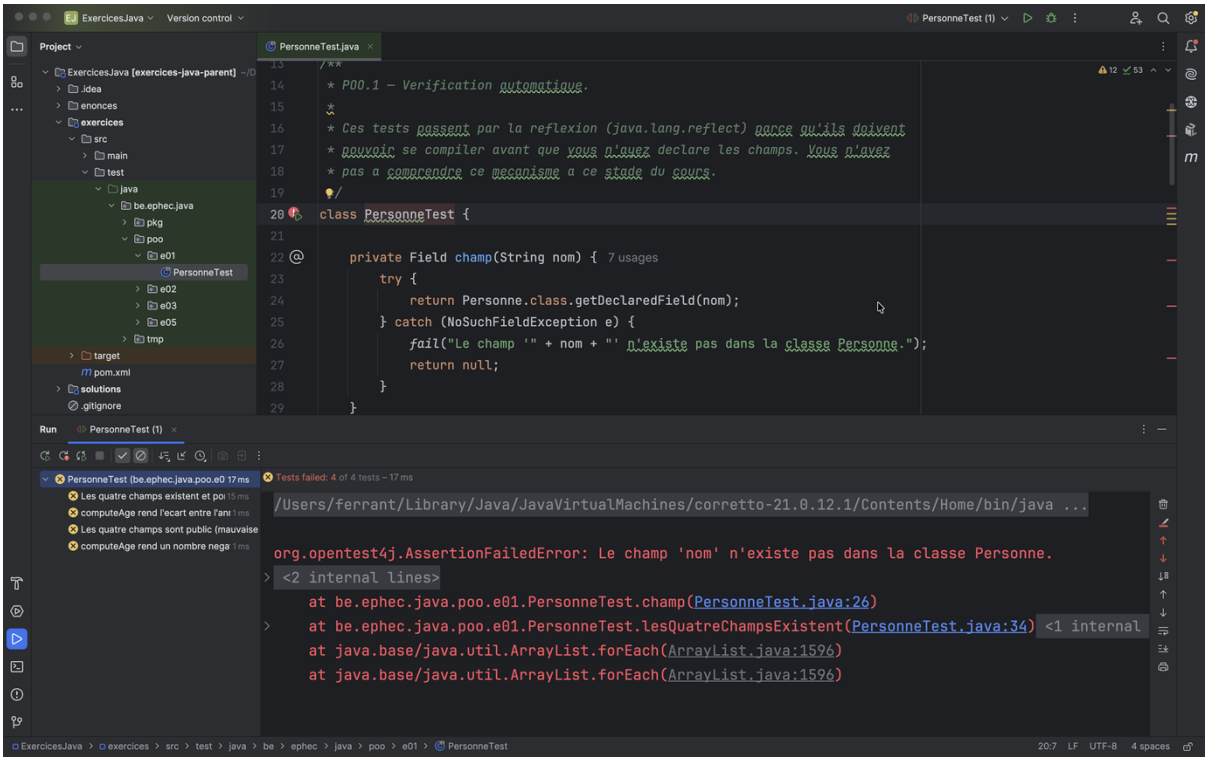
Si votre programme lit au clavier avec un `Scanner`, c'est dans cette fenêtre Run que vous tapez : cliquez dedans avant de saisir.

IDE.17 — Lancer les tests et lire le rapport

C'est le geste que vous répéterez le plus souvent de toute l'année, et celui qui vous dira si votre travail est correct.

Ouvrez la classe de test indiquée en fin d'énoncé. Une double flèche verte apparaît dans la marge à hauteur du nom de la classe : elle lance tous les tests du fichier. Une flèche simple, en face de chaque méthode, n'en lance qu'un. Le raccourci est `Ctrl+Maj+F10` / `^⇧R`.

Au départ, tout échoue. C'est normal : vous n'avez encore rien écrit.

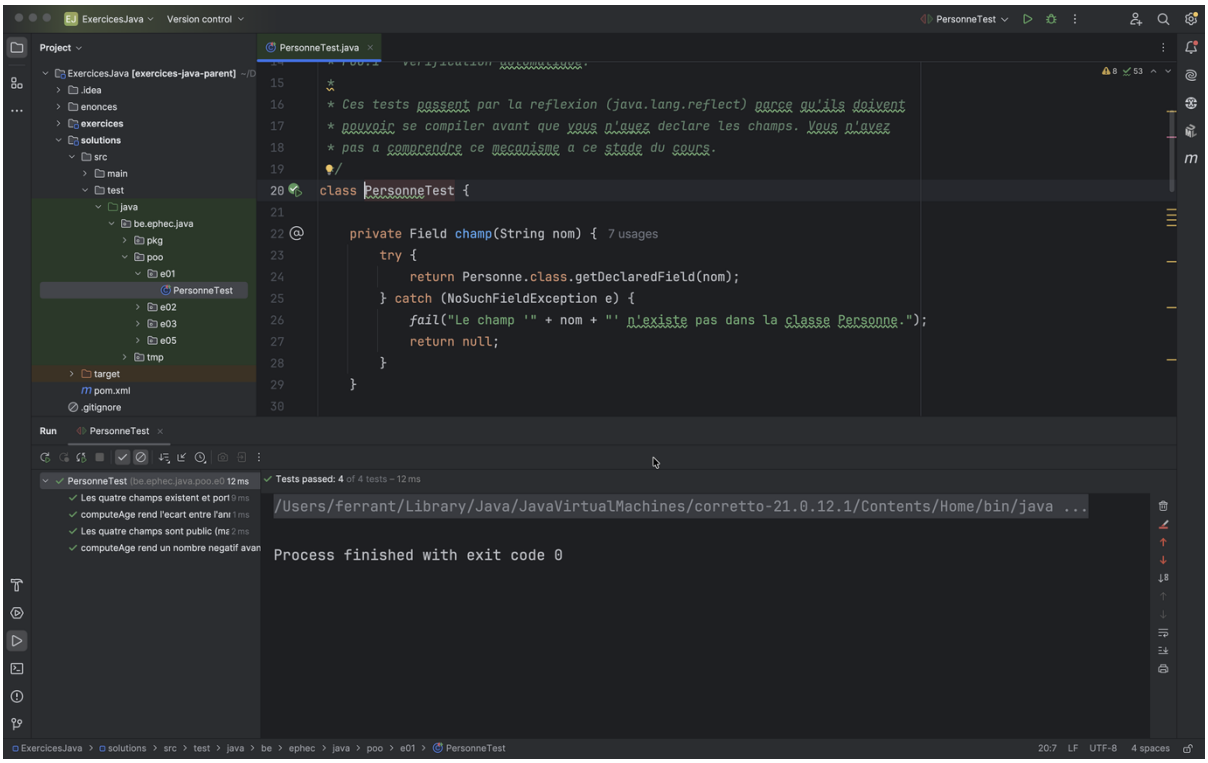


Quatre tests sur quatre échouent : l'état initial.

Le panneau se lit en trois temps. À gauche, la liste des tests, chacun avec sa croix ou sa coche. En haut, le compte. À droite, pour le test sélectionné, la raison de l'échec.

Lisez cette raison. Les messages sont écrits pour vous : ici, « Le champ 'nom' n'existe pas dans la classe Personne » vous dit exactement quoi faire. Se contenter de voir la croix rouge et retourner au code à l'aveugle vous fera perdre bien plus de temps que de lire trois lignes.

Quand tout est correct, l'ensemble passe au vert.



Le même fichier de test, une fois le travail fait.

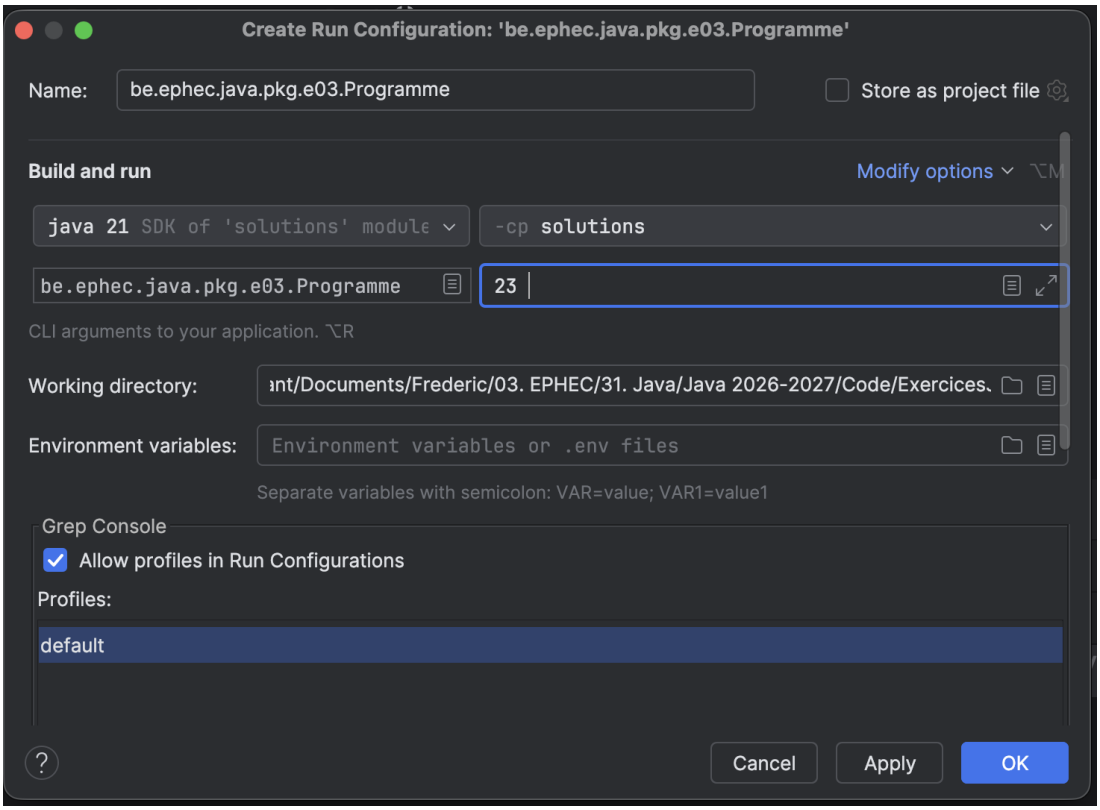
Pourquoi les deux captures ne montrent pas le même module

La première est prise dans le module exercices, la seconde dans solutions. Les deux modules contiennent les mêmes classes de test, à l'identique : le corrigé subit exactement les tests que vous subissez, il n'en a pas de plus indulgents.

Vous obtiendrez le second résultat sur votre propre code, dans le module exercices.

IDE.18 — Passer des arguments au programme

La méthode `main` reçoit un tableau de chaînes, `String[] args`. Pour le remplir, il faut passer par la configuration de lancement : `Run → Edit Configurations`, champ `Program arguments`.



Le champ Program arguments d'une configuration de lancement.

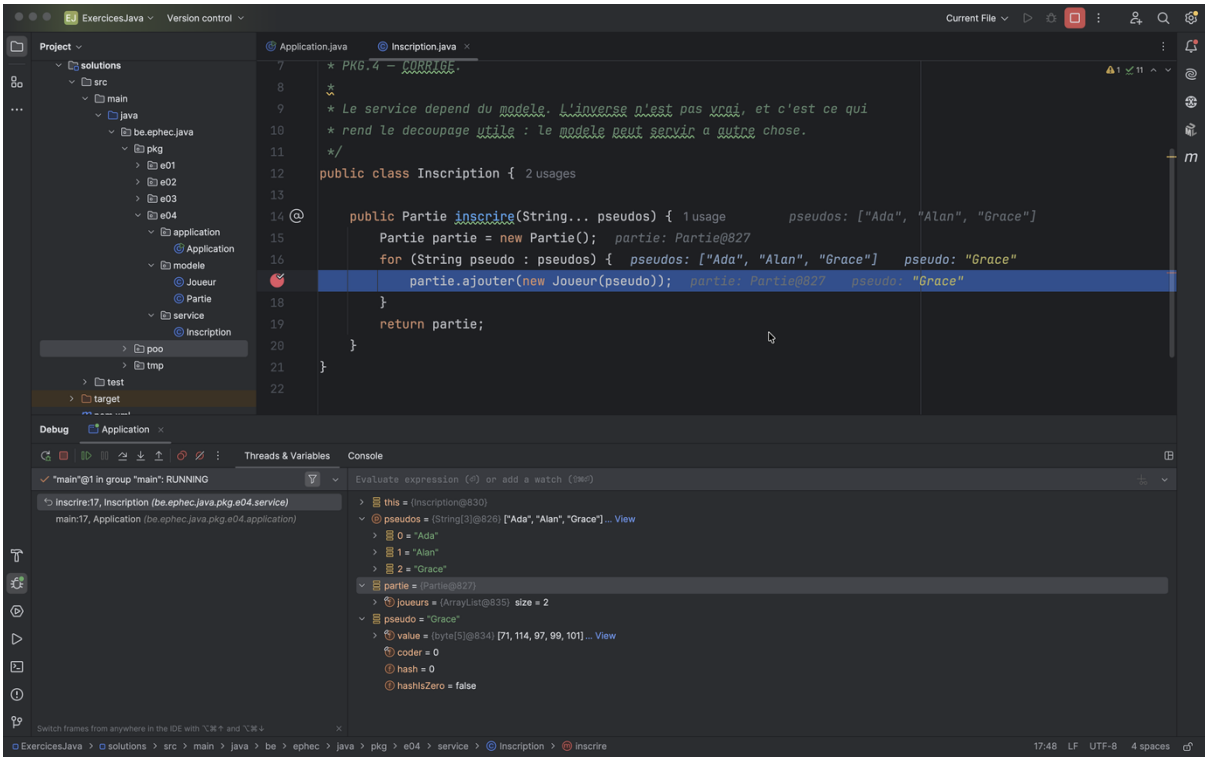
Les arguments sont séparés par des espaces. Pour en passer un qui contient une espace, entourez-le de guillemets.

IDE.19 — Le débogueur

Le débogueur suspend le programme à l'endroit de votre choix et vous laisse regarder ce qui s'y passe. C'est de loin le meilleur moyen de comprendre ce que fait vraiment votre code — et bien plus efficace que de semer des affichages un peu partout.

Le poser

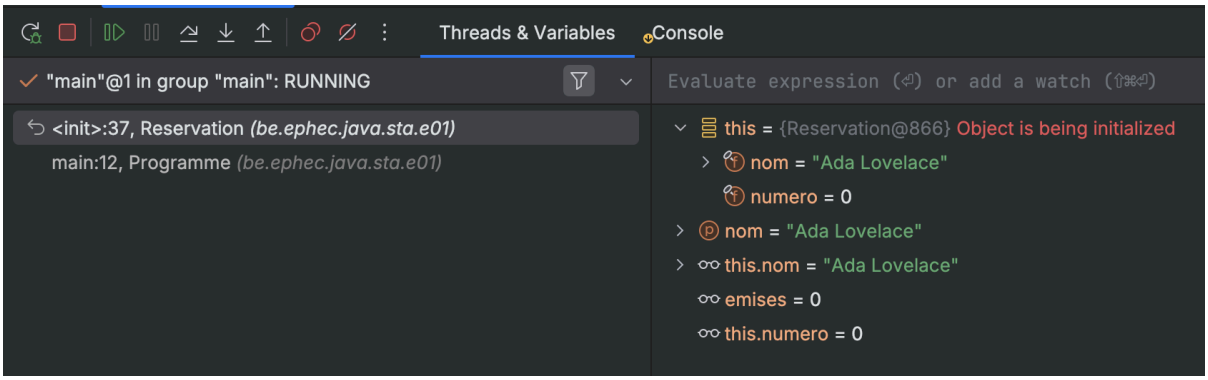
Cliquez dans la marge, à gauche d'une ligne : un point rouge apparaît. C'est un point d'arrêt. Lancez ensuite le programme avec l'insecte plutôt qu'avec la flèche verte. L'exécution s'arrête sur cette ligne, avant de l'exécuter.



Arrêt au troisième tour de boucle.

Le lire

- **Frames**, à gauche : la pile des appels. Sur la capture, `inscrire` a été appelée depuis `main`. C'est la pile dont parle le mémento technique, rendue visible.
- **Variables**, à droite : tout ce qui existe à cet instant. Les objets se déplient et montrent leur contenu — ici, `partie` contient une liste de deux joueurs, ce qui confirme qu'on est au troisième tour.
- Les annotations grises dans l'éditeur affichent la valeur des variables directement à côté du code.
- **Evaluate expression**, en haut du panneau Variables : tapez une expression — `Reservation.emises`, `liste.size()` — pour voir sa valeur à l'instant de l'arrêt. C'est utile pour ce que le panneau n'affiche pas de lui-même.



Arrêt dans le constructeur de Reservation : nom et numero sont dans `this`, le compteur `static` emises est affiché à part.

Un champ `static` n'apparaît pas quand on déplie `this` : il n'appartient pas à l'objet. IntelliJ l'affiche à part quand la ligne d'arrêt l'utilise ; sinon, demandez-le dans Evaluate expression.

Avancer

Action	Windows / Linux	macOS
Ligne suivante, sans entrer dans les appels	F8	F8
Entrer dans la méthode appelée	F7	F7
Sortir de la méthode courante	Maj+F8	⇧F8
Reprendre jusqu'au prochain arrêt	F9	⌥⌘R

Action	Windows / Linux	macOS

Trois choses qui surprennent au début

Au début d'un constructeur, les champs valent encore 0 ou `null`, et IntelliJ annonce `Object is being initialized`. Avancez avec F8 pour les voir se remplir.

Les objets portent un numéro — Partie@827, `ArrayList`@835. Ce numéro n'a aucune signification en soi et change à chaque exécution : il sert seulement à distinguer deux objets l'un de l'autre. Deux variables qui affichent le même numéro désignent le même objet, et c'est une observation précieuse.

Déplier une chaîne de caractères montre ses entrailles — un tableau d'octets, un champ hash. Ignorez-les : c'est la mécanique interne de `String`, pas quelque chose que vous ayez à comprendre.

F — Transformer du code sans le casser

Renommer une classe, déplacer une méthode, extraire un bout de code : ces opérations sont mécaniques et IntelliJ les fait à votre place, en corrigeant partout où c'est nécessaire. Les faire à la main, c'est se garantir un oubli.

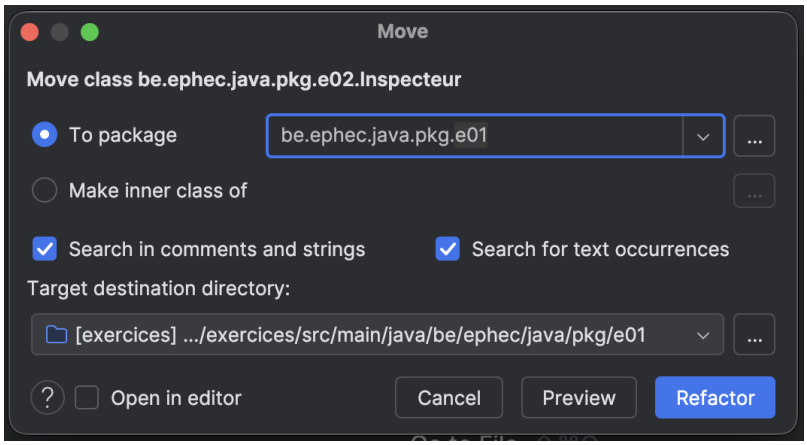
IDE.20 — Renommer

Curseur sur le nom, puis `Maj+F6`. Tapez le nouveau nom et validez : IntelliJ met à jour toutes les utilisations, dans tout le projet, y compris le nom du fichier si c'est une classe.

Cela vaut pour tout : classes, méthodes, variables, paramètres, packages. Il n'y a donc aucune raison de conserver un mauvais nom par crainte du travail de correction — voir `CC-NOM.2` du mémento Clean Code, qui repose entièrement sur cette possibilité.

IDE.21 — Déplacer une classe

Sélectionnez la classe dans la fenêtre Project, puis `F6`. Indiquez le package de destination.



La boîte Move, avec le package de destination.

IntelliJ déplace le fichier, corrige sa déclaration de `package`, et ajoute les `import` nécessaires dans toutes les classes qui l'utilisaient.

Déplacer, pas copier-coller

La tentation est de créer le nouveau package, d'y coller le fichier et de supprimer l'ancien. Cela marche parfois, et donne parfois deux classes du même nom dans deux packages, ce qui produit des erreurs incompréhensibles. Voir `ERR.5`.

L'exercice `PKG.4` est bâti sur ce geste, et sa classe de test vérifie explicitement que rien ne subsiste à l'ancien emplacement.

IDE.22 — Extraire une méthode

Sélectionnez les lignes à extraire, puis `Ctrl+Alt+M` / `⌥⌘M`. IntelliJ crée une méthode contenant ces lignes, détermine seul les paramètres nécessaires et la valeur de retour, et remplace les lignes d'origine par un appel.

C'est l'outil du mémento Clean Code : `CC-MET.2` demande des méthodes courtes, `CC-MET.3` qu'une méthode ne fasse qu'une chose, et `CC-COM.1` de remplacer un commentaire par une méthode bien nommée. Les trois se font avec ce raccourci.

G — Naviguer

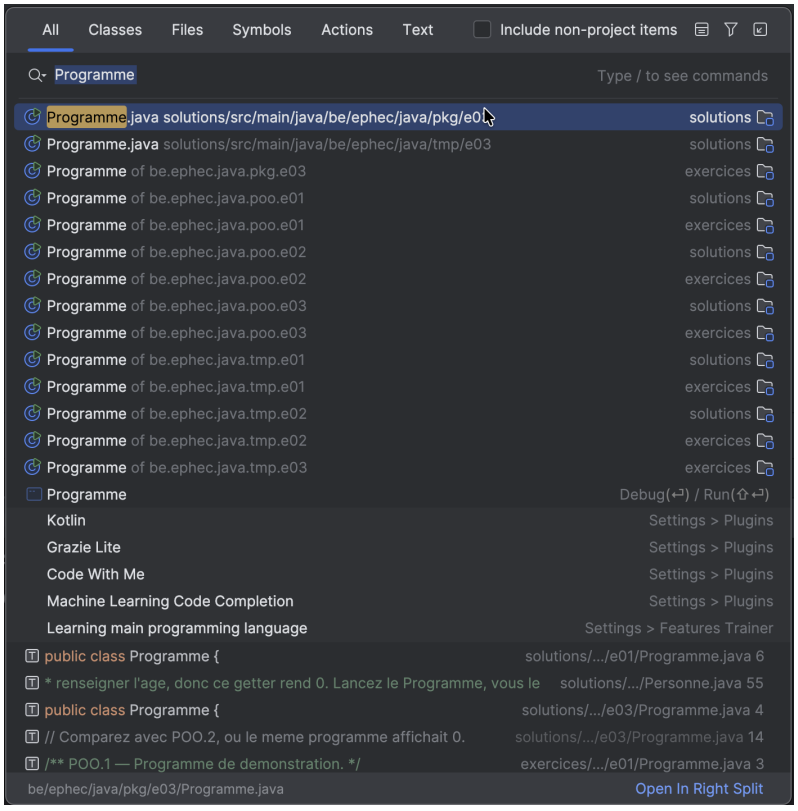
IDE.23 — Aller à la déclaration, et revenir

`Ctrl+clic` / `⌘clic` sur n'importe quel nom saute à sa déclaration. Cela fonctionne sur vos classes comme sur celles du JDK : vous pouvez lire le code source de `ArrayList` en un clic.

Pour revenir d'où vous venez : `Ctrl+Alt+←` / `⌘[`. C'est le bouton « page précédente » de l'éditeur, et il mémorise tout un historique.

IDE.24 — Chercher n'importe quoi

Appuyez deux fois sur `Maj`. Une barre de recherche s'ouvre et cherche partout à la fois : classes, fichiers, symboles, et jusqu'aux commandes d'IntelliJ.



Search Everywhere.

C'est le raccourci à retenir si vous n'en retenez qu'un. Vous cherchez une classe dont vous ne savez plus dans quel package elle vit, ou un réglage dont vous ne savez plus dans quel menu il se cache : les deux se trouvent ici.

IDE.25 — La structure d'un fichier

`Ctrl+F12` / `⌘F12` affiche la liste des champs et méthodes du fichier courant. Tapez quelques lettres pour filtrer, validez pour y sauter.

Utile dès qu'une classe dépasse l'écran — ce qui, si vous suivez le mémento Clean Code, ne devrait pas arriver souvent.

H — Raccourcis et réglages

IDE.26 — Les raccourcis à connaître

IntelliJ en compte plusieurs centaines. Voici ceux qui servent réellement, regroupés par usage. Apprenez-en deux ou trois par semaine plutôt que la liste entière.

Écrire

Action	Windows / Linux	macOS
Compléter	`Ctrl+Espace`	`^Espace`
Corriger, importer, proposer	`Alt+Entrée`	`⌥⏎`
Générer (accesseurs, constructeur)	`Alt+Inser`	`⌘N`
Reformater le code	`Ctrl+Alt+L`	`⌥⌘L`
Dupliquer la ligne	`Ctrl+D`	`⌘D`
Déplacer la ligne	`Alt+Maj+↑ ↓`	`⌥⇧↑ ↓`
Commenter la sélection	`Ctrl+/`	`⌘/`
Coller un contenu précédent	`Ctrl+Maj+V`	`⇧⌘V`

Naviguer

Action	Windows / Linux	macOS
Chercher partout	`Maj Maj`	`⇧⇧`
Aller à la déclaration	`Ctrl+clic`	`⌘clic`
Revenir en arrière	`Ctrl+Alt+←`	`⌘[`
Structure du fichier	`Ctrl+F12`	`⌘F12`
Fichier récent	`Ctrl+E`	`⌘E`

Exécuter

Action	Windows / Linux	macOS
Relancer la dernière exécution	`Maj+F10`	`^R`
Lancer ce qui est sous le curseur	`Ctrl+Maj+F10`	`^⇧R`
Déboguer la dernière configuration	`Maj+F9`	`^D`
Ligne suivante en débogage	`F8`	`F8`

Transformer

Action	Windows / Linux	macOS
Renommer	`Maj+F6`	`⇧F6`
Déplacer	`F6`	`F6`
Extraire une méthode	`Ctrl+Alt+M`	`⌥⌘M`
Extraire une variable	`Ctrl+Alt+V`	`⌥⌘V`

Si vous ne devez en retenir que deux : `Maj Maj` pour trouver n'importe quoi, et `Alt+Entrée` / `⌥⏎` pour qu'IntelliJ propose la correction de ce qu'il vient de signaler.

IDE.27 — Plugins et apparence

`Settings → Plugins` sous Windows, `IntelliJ IDEA → Settings → Plugins` sous macOS. Deux méritent le détour.

- **Key Promoter X** affiche le raccourci correspondant chaque fois que vous faites quelque chose à la souris. C'est le moyen le plus efficace d'apprendre les raccourcis : ils arrivent au moment où ils serviraient.
- **PlantUML integration** affiche les diagrammes de classes utilisés dans certains énoncés.

L'apparence se règle dans `Settings → Appearance`. Thème clair ou sombre, taille de la police de l'éditeur : réglez-les à votre confort, et augmentez la police si vous devez projeter votre écran.

IDE.28 — JavaFX : rien à configurer, ou presque

Vous trouverez en ligne des procédures expliquant qu'il faut télécharger un SDK `JavaFX`, l'installer, puis indiquer à IntelliJ où il se trouve. Ne les suivez pas pour ce cours : elles concernent les projets sans Maven.

`JavaFX` ne fait plus partie du JDK depuis Java 11 — voir `TEC-FX.2` du mémento technique. C'est donc une bibliothèque comme une autre, et le projet du cours la déclare dans son `pom.xml` au même titre que `JUnit` :

```
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>21.0.2</version>
</dependency>
```

Maven la télécharge au premier chargement et la place sur le classpath. Vous n'avez rien à installer, rien à désigner, rien à régler.

Une seule chose peut être ajoutée, et elle est facultative : Scene Builder, l'éditeur visuel de fichiers `.fxml`. Voir `IDE.29`.

Lancer une application JavaFX

C'est le seul point où `JavaFX` se distingue vraiment. Une application `JavaFX` a besoin que la bibliothèque soit annoncée à la JVM au démarrage, ce qu'un simple clic sur la flèche verte ne fait pas toujours. Deux façons de procéder, selon ce que l'énoncé indique.

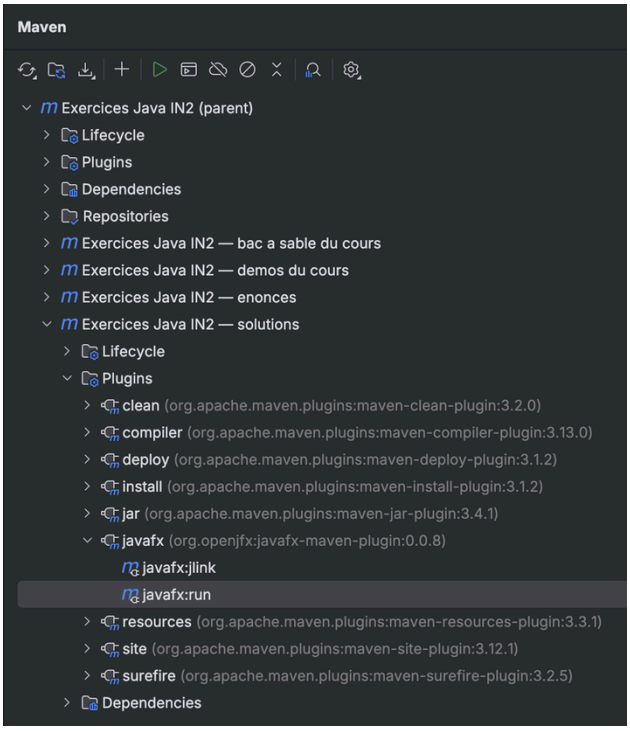
- **Depuis IntelliJ**, en lançant une classe de démarrage distincte de la classe d'application : le `Lanceur`. C'est le montage que le projet du cours utilise, et il est expliqué le moment venu dans le fascicule `FX`.
- **Depuis Maven**, par le greffon `JavaFX`, qui construit lui-même la ligne de commande correcte. C'est la solution de secours quand le lancement normal échoue : la procédure suit.

Lancer par Maven : javafx:run

1. Ouvrez la fenêtre Maven. Cliquez sur l'icône `m` de la barre verticale de droite, ou passez par `View → Tool Windows → Maven`.

2. Dépliez le bon module. La fenêtre montre le projet parent, puis ses modules, chacun nommé « Exercices Java IN2 — … » : *enonces* (vos exercices), *solutions*, *demos du cours* et *bac a sable du cours*. Dépliez celui qui contient la classe à lancer, en général *enonces*. Pas le nœud du haut, marqué « (parent) » : depuis lui, Maven lancerait la commande dans chaque module, l'un après l'autre.

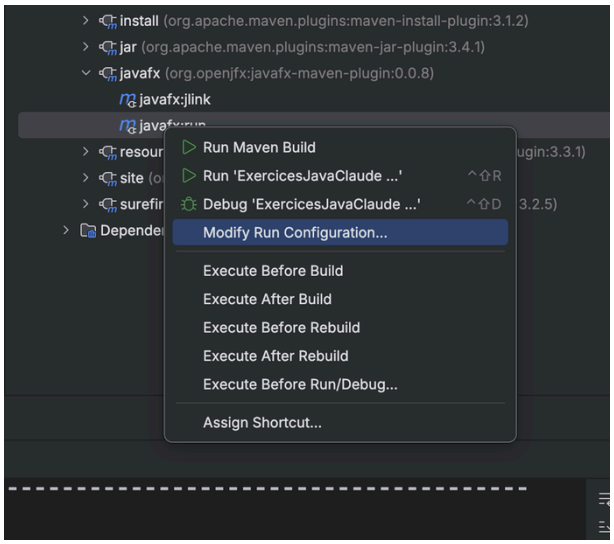
3. Lancez. Sous le module, dépliez `Plugins → javafx`, puis double-cliquez sur `javafx:run`. La console Maven s'ouvre en bas, puis la fenêtre de l'application apparaît ; la fermer termine l'exécution. La première fois, Maven télécharge le greffon : il faut une connexion internet, et quelques secondes de patience.



La fenêtre Maven : le greffon javafx, sous Plugins, dans le module à lancer.

Quelle classe est lancée ? Celle que désigne la propriété `javafx.mainClass` dans le `pom.xml` du module : pour les énoncés et les solutions, le premier exercice `JavaFX`, `be.ephec.java.fx.e01.CompteurParticipants`. Le greffon accepte directement la classe d'application : inutile de passer par le Lanceur.

Lancer un autre exercice, sans toucher au `pom.xml` : faites un clic droit sur `javafx:run` et choisissez `Modify Run Configuration…`.



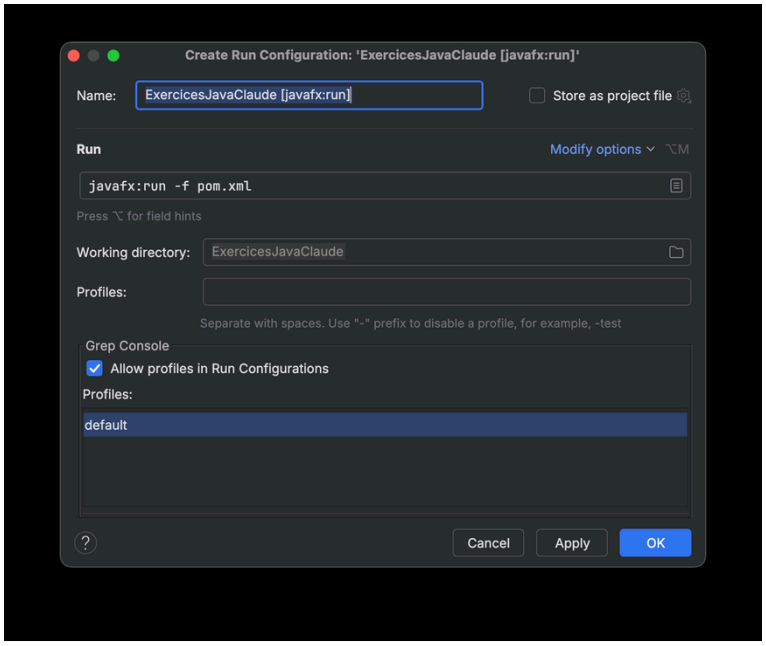
Le menu contextuel de javafx:run.

Dans la fenêtre qui s'ouvre, trois réglages :

- **Name** : un nom parlant, « `FX.2` » par exemple.
- **Run** : la commande. Gardez ce qu'IntelliJ a écrit, et ajoutez la classe à lancer après `javafx:run` :

```
javafx:run -Djavafx.mainClass=be.ephec.java.fx.e02.CompteurParticipants -f pom.xml
```

- **Working directory** : le dossier dont Maven lit le `pom.xml`. Vérifiez qu'il désigne bien le module, par exemple le dossier `exercices`. S'il indique la racine du projet, choisissez le dossier du module avec l'icône de dossier : sinon, Maven partirait du `pom.xml` parent et lancerait la commande dans chaque module.



La configuration d'exécution : son nom, la commande Maven, le dossier de travail.

Validez par OK. La configuration apparaît ensuite dans la liste déroulante en haut à droite d'IntelliJ, et la flèche verte la relance.

Si vous obtenez le message `JavaFX runtime components are missing`, c'est exactement ce problème : voir ERR.7.

IDE.29 — Installer Scene Builder

Scene Builder est un éditeur visuel de fichiers `.fxml` : on y compose la fenêtre en déposant les composants à la souris, et il écrit le XML correspondant. Il est gratuit.

Ce n'est pas un greffon d'IntelliJ, mais un logiciel séparé. IntelliJ ne fait que lui passer la main quand vous ouvrez un fichier FXML. Il faut donc l'installer, puis dire à IntelliJ où il se trouve — deux étapes distinctes, et c'est la seconde qu'on oublie.

1. Télécharger

Sur `gluonhq.com/products/scene-builder`, en prenant la version correspondant à votre système. Sur un Mac Apple Silicon, vérifiez qu'il s'agit bien de la variante `aarch64` et non `x64`.

Quelle version. Les numéros de Scene Builder suivent ceux de `JavaFX`. Le cours utilise `JavaFX` 21 : prenez Scene Builder 21, ou la version disponible la plus proche. Un Scene Builder plus récent que votre `JavaFX` peut écrire dans le FXML des attributs que votre version ne comprend pas — le fichier s'ouvre alors correctement dans l'éditeur et échoue au lancement.

2. Installer

Installation ordinaire : un .dmg à glisser dans Applications sous macOS, un installateur sous Windows. Aucun réglage à faire pendant l'installation.

Notez le chemin où il atterrit, vous en aurez besoin à l'étape suivante. Par défaut :

```
macOS     /Applications/SceneBuilder.app
Windows   C:\Users\<vous>\AppData\Local\SceneBuilder\SceneBuilder.exe
```

3. Le déclarer dans IntelliJ

`Settings → Languages & Frameworks → JavaFX`, champ `Path to SceneBuilder`. Désignez le fichier repéré ci-dessus.

C'est la seule ligne à renseigner. Tant qu'elle est vide, IntelliJ ouvre les fichiers FXML comme du XML ordinaire, sans onglet d'édition visuelle — et rien n'indique qu'il manque quelque chose.

4. Vérifier

Ouvrez n'importe quel fichier `.fxml` du projet. Deux onglets doivent apparaître en bas de l'éditeur : `Text` et `Scene Builder`. Le second affiche la fenêtre telle qu'elle sera, avec la palette de composants sur le côté.

Si l'onglet n'apparaît pas, le chemin est erroné ou pointe vers le dossier plutôt que vers l'exécutable.

S'en servir, ou pas

Scene Builder fait gagner du temps sur les fenêtres complexes, où placer les composants à la main devient fastidieux. Il rend aussi visible ce qu'un gestionnaire de placement fait réellement, ce qui aide à comprendre pourquoi une fenêtre se réorganise quand on la redimensionne.

Mais vous pouvez très bien vous en passer, et je vous conseille de commencer sans. Écrire quelques fichiers FXML à la main apprend leur structure, et cette structure est ce que l'examen évalue — pas votre habileté à la souris.

> **Deux attributs à ne jamais perdre de vue**
>
> Un fichier FXML contient un attribut fx:controller sur son élément racine, qui désigne la classe contrôleur, et des attributs fx:id sur les composants que le contrôleur manipule. Ce sont eux qui relient la vue au code.
>
> Scene Builder les gère, mais ils se règlent dans un panneau discret. Si votre application se lance avec une fenêtre vide, ou lève une exception au démarrage en évoquant un champ `null`, vérifiez d'abord ces deux attributs dans l'onglet Text.

Annexe — Erreurs et solutions

Les messages les plus fréquents, leur cause, et ce qu'il faut faire. Cherchez le message que vous avez à l'écran plutôt que de lire l'annexe en entier.

ERR.1 — JDK isn't specified for module

```
java: JDK isn't specified for module 'solutions'
```

Un module Maven ne définit pas son propre JDK : il hérite de celui du projet. Ce message signifie donc presque toujours qu'aucun JDK n'est déclaré au niveau du projet, et il nomme simplement le premier module qu'IntelliJ a essayé de compiler.

`File → Project Structure → Project`, champ SDK : choisissez un JDK 21. S'il n'y en a aucun dans la liste, voir `IDE.5`.

Si le SDK du projet est correct et que le message persiste, vérifiez le module lui-même : `Project Structure → Modules`, onglet `Dependencies`, champ `Module SDK`. Il doit indiquer « Project SDK » et non un JDK nommé.

En dernier recours, fermez le projet, supprimez le dossier `.idea` et les fichiers `.iml`, puis rouvrez par le `pom.xml`. Vous ne perdez rien : ce dossier ne contient que des préférences d'affichage.

ERR.2 — Cannot resolve symbol

```
java: cannot find symbol
  symbol:   class Test
  location: package org.junit.jupiter.api
```

Une classe venant d'une bibliothèque est introuvable. Le plus souvent, IntelliJ n'a pas encore chargé ce que déclare le `pom.xml` — après un premier chargement interrompu, ou après avoir déplié une nouvelle archive dans le projet.

Ouvrez la fenêtre Maven et cliquez sur le bouton de rechargement. Voir `IDE.9`. Si le téléchargement échoue, vérifiez votre connexion : Maven va chercher les bibliothèques sur Internet à la première utilisation.

ERR.3 — Tout est rouge sans raison

Des dizaines de classes soulignées en rouge alors que le code n'a pas changé, souvent après une mise à jour d'IntelliJ ou l'ajout de fichiers dans le projet. Les index de résolution sont désynchronisés.

`File → Invalidate Caches… → Invalidate and Restart`. IntelliJ reconstruit ses index au redémarrage, ce qui prend une minute. Rien n'est perdu : ni votre code, ni vos réglages.

ERR.4 — Pas de flèche verte sur une classe de test

Trois causes possibles, dans l'ordre de fréquence.

- Le fichier n'est pas sous `src/test/java` mais sous `src/main/java`. Déplacez-le avec `F6`.
- La déclaration de `package` ne correspond pas à l'emplacement du fichier dans l'arborescence. Les deux doivent coïncider exactement.
- `JUnit` n'est pas chargé : voir `ERR.2`.
- Aucune classe du projet n'a de flèche verte, pas seulement les tests : voir `ERR.11`.

ERR.5 — Deux classes du même nom

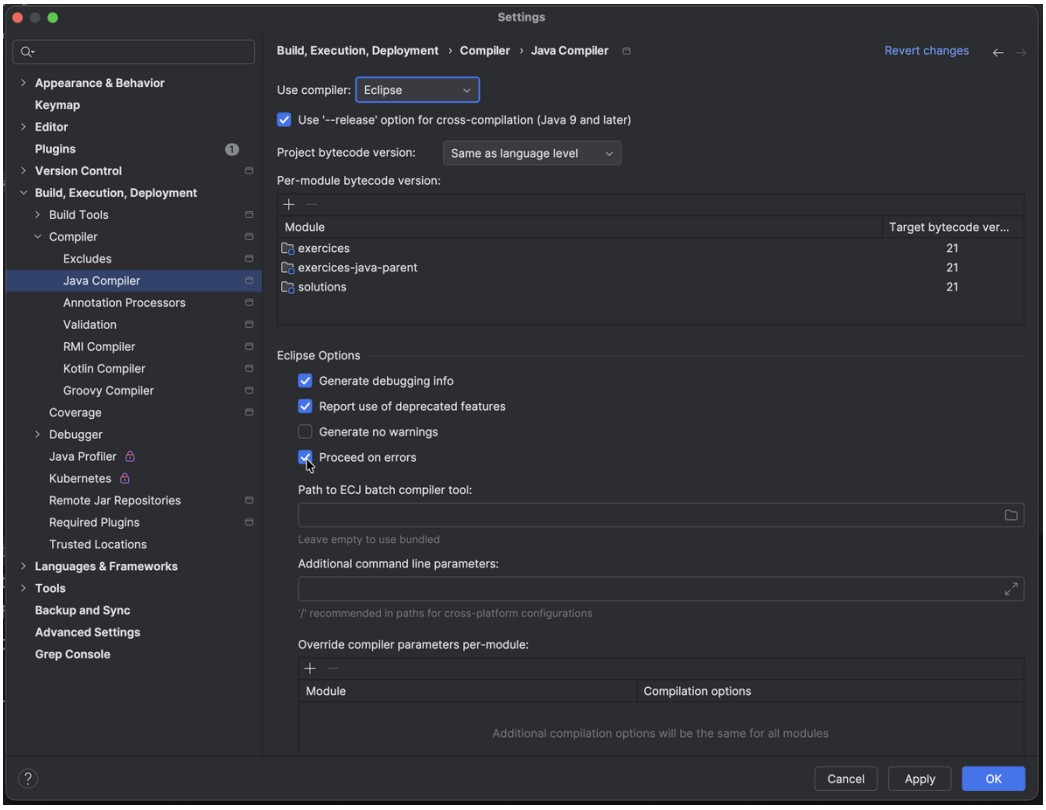
Des erreurs incompréhensibles après une réorganisation : une méthode existe pourtant, un `import` semble correct mais ne l'est pas. Presque toujours, une classe a été copiée-collée dans un nouveau package sans être supprimée de l'ancien. Vous en modifiez une, le programme utilise l'autre.

Cherchez le nom de la classe avec `Maj Maj` : si deux résultats apparaissent dans deux packages différents, vous tenez le coupable. Supprimez celle qui est à la mauvaise place, et à l'avenir déplacez avec `F6` au lieu de copier — voir `IDE.21`.

ERR.6 — Lancer malgré une erreur de compilation

Par défaut, IntelliJ compile tout le module avant de lancer quoi que ce soit. Une erreur dans une classe empêche donc de lancer une autre classe, même sans rapport avec elle.

Il est possible de contourner ce comportement : `Settings → Build, Execution, Deployment → Compiler → Java Compiler`, basculez `Use compiler` sur `Eclipse`, puis cochez `Proceed on errors`. L'option n'apparaît qu'après avoir changé de compilateur.



L'option Proceed on errors, disponible avec le compilateur Eclipse.

> **Ne le faites pas dans ce projet**
>
> Le compilateur Eclipse embarqué dans IntelliJ 2024.3 ne gère Java que jusqu'à la version 18. Ce cours utilise Java 21 : dès que vous basculez dessus, plus rien ne compile, et vous obtenez le message décrit en `ERR.8`.
>
> L'option est donc inutilisable ici, et c'est aussi bien : changer de compilateur pour contourner une erreur de compilation est un pis-aller. Le bon réflexe reste de lire le message et de corriger.
>
> Dans le projet du cours, le problème que l'option prétend résoudre ne se pose de toute façon pas : les squelettes fournis compilent toujours, puisqu'ils lèvent une exception au lieu d'être vides.

ERR.7 — JavaFX runtime components are missing

```
Error: JavaFX runtime components are missing,
and are required to run this application
```

La bibliothèque `JavaFX` est bien présente dans le projet, mais elle n'a pas été annoncée à la JVM au démarrage. Ce n'est pas une dépendance manquante : c'est une histoire de mode de lancement.

Lancez par `javafx:run` depuis la fenêtre Maven, ou par la classe de démarrage prévue par l'énoncé. Voir `IDE.28`.

L'explication de fond — pourquoi `JavaFX` est sortie du JDK et ce que cela change — se trouve en `TEC-FX.2` du mémento technique.

ERR.8 — source level should be in ... : 21

```
java: java.lang.IllegalArgumentException: source level should be in
'1.1'...'1.8','9'...'18' (or '5.0'..'18.0'): 21
```

Ce message ne vient pas de `javac` mais du compilateur Eclipse. Il dit qu'on lui demande de compiler en Java 21 alors qu'il ne connaît le langage que jusqu'à la version 18.

La cause est presque toujours la même : le compilateur du projet a été basculé sur Eclipse, souvent pour accéder à l'option décrite en `ERR.6`. Une fois basculé, il le reste.

Le remède. `Settings → Build, Execution, Deployment → Compiler → Java Compiler`, champ `Use compiler` : remettez-le sur `Javac`. Relancez ensuite la compilation.

Si le message persiste, vérifiez dans le même écran que le champ « Project bytecode version » et les « Target bytecode version » par module indiquent 21 ou « Same as language level ».

> **Un message qui désigne le coupable**
>
> La liste des versions acceptées est signée : elle s'arrête à 18, ce qui ne correspond à aucun JDK que vous auriez installé. C'est le genre de détail qui permet d'identifier l'outil fautif sans chercher plus loin.
>
> À l'inverse, javac dirait quelque chose comme « release version 21 not supported », en nommant la version qu'il connaît.

ERR.9 — Un exercice inachevé bloque tous les autres

Vous laissez un exercice à moitié écrit, avec une accolade manquante ou un type qui ne correspond pas. Et soudain plus rien ne se lance — pas même un exercice fait la semaine dernière, qui n'a aucun rapport.

Ce n'est pas une panne, c'est ainsi que Java compile. Tout le dossier `src/main/java` du module se compile d'un seul bloc, et `src/test/java` en dépend. Un seul fichier fautif suffit donc à empêcher la production de tous les autres.

Ce qui n'est PAS bloqué

Le projet compte trois modules Maven indépendants — `exercices`, `solutions` et `demos` — et aucun ne dépend des deux autres. Une erreur dans votre code n'empêche donc ni d'ouvrir un corrigé, ni de lancer une démonstration du cours.

Trois façons de s'en sortir

1. Commenter le code fautif. Sélectionnez les lignes en chantier et faites `Ctrl+/` / `⌘/`. Tout recompile, vous reprenez votre travail plus tard. C'est la solution en dix secondes, et celle à prendre comme réflexe.

2. Remettre le squelette d'origine. Extrayez l'archive de la séance dans un dossier temporaire — et non par-dessus votre projet, voir `TEC-ZIP.1` — puis recopiez le seul fichier concerné. Vous perdez ce que vous aviez écrit dans ce fichier-là, rien d'autre : `TEC-ZIP.2` du mémento technique explique pourquoi.

3. Demander à IntelliJ de lancer malgré tout. Dans `Run → Edit Configurations`, section `Before launch`, remplacez la tâche `Build` par `Build, no error check`. IntelliJ compile alors ce qu'il peut et lance votre classe si elle-même est correcte.

> **La troisième solution a ses limites**
>
> Elle ne fonctionne que si la classe que vous lancez a réellement compilé. Si c'est elle qui contient l'erreur, rien ne la sauvera.
>
> Et elle se règle configuration par configuration, ce qui devient vite fastidieux. Considérez-la comme un dépannage ponctuel, pas comme un réglage à laisser en place.
>
> Ne cherchez surtout pas à obtenir le même effet en changeant de compilateur : voir `ERR.6`, qui explique pourquoi cela ne marche pas dans ce projet.

Ce qu'il faut en retenir

Laissez votre code dans un état qui compile avant de passer à autre chose. Un exercice incomplet mais compilable ne gêne personne ; un exercice à moitié tapé bloque toute la soirée.

C'est d'ailleurs pourquoi les squelettes qu'on vous fournit lèvent une `UnsupportedOperationException` au lieu d'être vides : un squelette compile dès le premier instant, avant même que vous n'ayez écrit quoi que ce soit.

ERR.10 — Un fichier disparaît de la fenêtre Project

Vous travaillez sur un fichier, et il s'efface de l'arborescence. Parfois le dossier entier. Il revient de lui-même au bout d'un moment, ou au prochain clic dans la fenêtre.

Votre fichier n'est pas perdu. Il est sur le disque, exactement là où vous l'avez laissé. C'est la *vue* qu'IntelliJ en a qui s'est désynchronisée.

Pourquoi cela arrive sur un disque réseau

IntelliJ ne relit pas le disque à chaque affichage : ce serait trop lent. Il garde une image de l'arborescence en mémoire, et demande au système d'exploitation de le prévenir quand un fichier change.

Ce mécanisme d'alerte ne fonctionne pas sur un partage réseau. IntelliJ le sait, et il l'annonce parfois lui-même dans une bulle au démarrage :

```
External file changes sync may be slow:
Project files cannot be watched (are they under network mount?)
```

Faute d'être prévenu, IntelliJ se rabat sur une relecture périodique. Entre deux relectures, sa vue peut être en retard. Et si le partage répond lentement au moment exact où il relit un dossier, la lecture échoue : IntelliJ conclut que le dossier est vide, et l'affiche vide. La relecture suivante, réussie, remet tout en place — d'où le retour spontané.

Ce qu'il faut faire

Forcez la relecture : `File → Reload All from Disk` — `Ctrl+Alt+Y` / `⌥⌘Y`. L'arborescence est reconstruite à partir du disque, et le fichier réapparaît.

Si vous voulez vous rassurer avant, ouvrez le dossier dans l'explorateur de fichiers du système : il y verra le fichier, lui.

> **Ce qu'il ne faut surtout pas faire**
>
> Recréer le fichier. Vous en obtiendriez un second, vide, qui écraserait le vôtre à l'enregistrement — et là, cette fois, le travail serait bel et bien perdu.
>
> Le réflexe est compréhensible, et c'est précisément pour cela qu'il faut l'avoir vu écrit avant de se trouver devant l'écran.

Le même symptôme, une autre cause

Si le fichier ne revient pas après un `Reload All from Disk`, ce n'est plus un problème d'affichage. Vérifiez alors qu'il n'a pas été déplacé par un `F6` involontaire, ou filtré par la barre de recherche de la fenêtre Project — quelques lettres tapées par erreur dans l'arborescence y font office de filtre, et masquent tout le reste.

En séance d'examen, signalez-le plutôt que d'improviser : cinq secondes de vérification valent mieux qu'un fichier écrasé.

ERR.11 — Les modules Maven ont disparu : plus rien ne se lance

Les dossiers `exercices`, `solutions` ou `demos` apparaissent comme des dossiers ordinaires : `src/main/java` n'est plus en bleu, aucune flèche verte ne s'affiche à côté des `main`, et les tests ne se lancent plus. Cela arrive souvent après avoir récupéré une nouvelle version du projet ou déplié une archive dedans : IntelliJ n'a pas relu les `pom.xml`.

IntelliJ propose normalement de le faire lui-même, par une petite icône Maven qui apparaît en haut à droite de l'éditeur. Elle est discrète, et on la manque facilement.

Ouvrez la fenêtre Maven et cliquez sur le bouton de rechargement. Voir `IDE.9`. Si la fenêtre Maven est vide, faites un clic droit sur le `pom.xml` à la racine du projet, puis `Add as Maven Project`. Si le problème persiste, voir `ERR.3`.

Avant de recharger, vérifiez que chaque module déclaré par le `pom.xml` racine existe bien, avec son propre `pom.xml` : s'il en manque un, Maven refuse de charger le projet entier, et tous les modules disparaissent d'un coup.

Annexe B — Mémento technique

Ce qu'IntelliJ fait à votre place

À quoi sert ce mémento

Vous allez écrire du Java toute l'année sans avoir besoin de savoir ce qui se passe entre le moment où vous cliquez sur la flèche verte et celui où le texte s'affiche à l'écran. Jusqu'au jour où quelque chose se passe mal — et là, sans ces notions, on cherche à l'aveugle.

Ce document répond aux questions que le cours ne pose pas directement. Que fait exactement le JDK. Pourquoi un programme Java tourne sur Windows comme sur Mac. Où vivent vos objets. Ce que Maven télécharge pendant la minute d'attente du premier lancement.

Il se consulte, il ne se lit pas d'une traite. Chaque section porte un code — `TEC-EXE.2` pour le tas, par exemple — et les énoncés d'exercices y renvoient au moment utile. Les codes sont cliquables.

Les deux autres documents

Ce mémento fait partie d'un ensemble de trois. Le mémento Clean Code rassemble les règles d'écriture. Le guide IntelliJ explique comment faire les choses dans l'outil.

La frontière est simple : ce qui est vrai de Java est ici, ce qui est propre à IntelliJ est dans le guide. Comprendre ce qu'est un JDK relève de ce document ; savoir où cliquer pour en changer relève du guide.

Sommaire

La plateforme Java	
`TEC-PLA.1`	JVM, JRE, JDK : qui fait quoi
`TEC-PLA.2`	Les trois éditions
`TEC-PLA.3`	Les versions et le support long terme
`TEC-PLA.4`	Que contient ma machine

Les interfaces graphiques	
`TEC-FX.1`	Trois générations, et pourquoi
`TEC-FX.2`	JavaFX ne fait plus partie du JDK
`TEC-FX.3`	Les modules Java de JavaFX
`TEC-FX.4`	Pourquoi une application JavaFX refuse de démarrer

De votre code au programme qui tourne	
TEC-CMP.1	Trois états de votre programme
TEC-CMP.2	Pourquoi Java tourne partout
TEC-CMP.3	Le classpath
TEC-CMP.4	Le dossier target

Ce qui se passe pendant l'exécution	
TEC-EXE.1	La pile
TEC-EXE.2	Le tas
TEC-EXE.3	Ce que contient vraiment une variable
TEC-EXE.4	Le ramasse-miettes

Maven	
TEC-MVN.1	À quoi sert Maven
TEC-MVN.2	Le pom.xml
TEC-MVN.3	Un projet, plusieurs modules Maven
TEC-MVN.4	Le cycle de vie
TEC-MVN.5	Le dépôt local

Recevoir les fichiers du cours	
TEC-ZIP.1	Extraire une archive au bon endroit
TEC-ZIP.2	Ne pas écraser son propre travail

Git, en deux mots	
TEC-GIT.1	Ce qu'est un gestionnaire de versions
TEC-GIT.2	Pourquoi certains dossiers sont ignorés
TEC-GIT.3	Le dépôt du fil rouge, et comment le parcourir

La plateforme Java

TEC-PLA.1 — JVM, JRE, JDK : qui fait quoi

Trois sigles, trois choses emboîtées l'une dans l'autre. Chacune contient la précédente.

Sigle	Nom	Contenu
`JVM`	Java Virtual Machine	Le programme qui exécute du bytecode Java
`JRE`	Java Runtime Environment	La JVM, plus la bibliothèque standard (`String`, `List`, `LocalDate`…)
`JDK`	Java Development Kit	Le JRE, plus les outils de développement : javac, javadoc, jar, jdb

```
JDK  =  JRE  +  javac, javadoc, jar, jdb, ...
        |
        +--  JVM  +  bibliotheque standard
```

La conséquence pratique tient en une phrase : pour exécuter un programme Java il suffit d'un JRE, pour en écrire un il faut un JDK. Comme vous écrivez du code, c'est un JDK qu'il vous faut.

> **Une correction par rapport à ce que vous lirez ailleurs**
>
> Beaucoup de schémas trouvés sur Internet présentent le JRE comme un produit qu'on télécharge séparément. C'était vrai jusqu'à Java 8. Depuis Java 11, le JRE n'est plus distribué seul : on installe un JDK, point. Le mot reste utile pour désigner la partie « exécution » de ce JDK, mais vous ne trouverez pas de JRE à télécharger.

TEC-PLA.2 — Les trois éditions

Java se décline en trois éditions, qui ne visent pas les mêmes machines.

Édition	Pour quoi	Vous concerne ?
Java SE	Standard. Applications de bureau, consoles, bibliothèques.	Oui — c'est ce cours
Jakarta EE	Entreprise. Applications serveur, web, transactions.	Plus tard, en spécialisation
Java ME	Micro. Systèmes embarqués, cartes à puce.	Non, et de plus en plus marginal

Quand une documentation parle de « Java » sans préciser, il s'agit presque toujours de Java SE.

Note de vocabulaire. L'édition entreprise s'appelait Java EE ou J2EE. Oracle en a transféré le développement à la fondation Eclipse à la fin des années 2010, et pour des raisons de marque elle a été renommée **Jakarta EE**. Vous rencontrerez les deux noms selon l'âge du document que vous lisez.

TEC-PLA.3 — Les versions et le support long terme

Depuis 2018, une nouvelle version de Java sort tous les six mois, en mars et en septembre. La plupart ne sont maintenues que six mois, jusqu'à la suivante.

Tous les deux ans environ, une version est désignée **LTS** — `Long` Term Support. Celle-là est maintenue pendant plusieurs années : elle reçoit des correctifs de sécurité longtemps après sa sortie. Les versions LTS récentes sont 8, 11, 17, 21 et 25.

Ce cours utilise Java 21. N'installez pas une version plus récente en pensant bien faire. Une version supérieure accepte des constructions que 21 ignore : votre code compilerait chez vous et refuserait de compiler ailleurs — sur la machine d'un condisciple, ou à l'examen.

Le projet du cours fixe d'ailleurs cette contrainte lui-même, dans son `pom.xml` : voir TEC-MVN.2.

TEC-PLA.4 — Que contient ma machine

Deux commandes, dans un terminal sous macOS ou Linux, dans une invite de commandes sous Windows.

```
java -version      # la version qui EXECUTE
javac -version     # la version qui COMPILE
```

La première doit répondre quelque chose comme :

```
openjdk version "21.0.12" 2024-07-16 LTS
OpenJDK Runtime Environment Corretto-21.0.12.7.1
OpenJDK 64-Bit Server VM Corretto-21.0.12.7.1 (build 21.0.12+7-LTS, mixed mode)
```

Si javac répond que la commande est introuvable alors que `java` fonctionne, vous avez un environnement d'exécution mais pas de kit de développement. Vous pouvez lancer des programmes Java, pas en compiler.

Si les deux répondent des versions différentes, plusieurs JDK cohabitent sur votre machine et ce n'est pas le même qui est trouvé en premier. Ça n'a rien d'anormal, mais il faut savoir lequel IntelliJ utilise — qui n'est d'ailleurs pas nécessairement celui de ces commandes.

Lister tous les JDK installés

Sous macOS :

```
/usr/libexec/java_home -V
```

Sous Windows, dans PowerShell :

```
Get-ChildItem "C:\Program Files\Java", "C:\Program Files\Eclipse Adoptium" -EA 0
```

C'est la commande à connaître le jour où IntelliJ vous annonce une version que vous ne vous souvenez pas d'avoir installée. Le réglage effectivement utilisé par IntelliJ, lui, se vérifie dans `Project Structure` : voir la section `IDE.6` du guide IntelliJ.

Les interfaces graphiques

TEC-FX.1 — Trois générations, et pourquoi

Java a connu trois bibliothèques successives pour construire des fenêtres. Elles ne se sont pas remplacées par mode : chacune corrige un défaut de la précédente.

Bibliothèque	Apparue	Principe
`AWT`	1995, avec Java 1.0	Chaque bouton est un vrai bouton du système d'exploitation
`Swing`	1998, avec Java 2	Java dessine lui-même tous les composants
`JavaFX`	2008, intégrée au JDK de 2012 à 2018	Description déclarative de l'interface, styles, écrans tactiles

AWT et sa limite

AWT confie l'affichage au système : un bouton AWT sous Windows est un bouton Windows, et sous macOS un bouton macOS. L'idée paraît bonne — l'application ressemble aux autres applications de la machine — mais elle impose de se limiter au dénominateur commun des trois systèmes, et les comportements diffèrent d'un poste à l'autre. Le même programme ne se comporte pas tout à fait pareil partout, ce qui est exactement ce que Java cherchait à éviter.

Swing et son principe

Swing prend le contre-pied : Java dessine lui-même chaque composant, pixel par pixel. L'apparence et le comportement deviennent identiques partout, au prix d'une interface qui ne ressemble à aucune autre sur la machine.

Swing introduit une organisation qu'on retrouvera à l'identique dans `JavaFX`, et qu'il vaut donc la peine de connaître :

- **les composants** — boutons, champs de texte, images ;
- **les conteneurs** — des composants qui en contiennent d'autres ;
- **les gestionnaires de placement** — à qui l'on confie la disposition, plutôt que de fixer des coordonnées, ce qui permet à la fenêtre de s'adapter quand on la redimensionne ;
- **les gestionnaires d'événements** — un clic devient un objet, transmis à du code qui décide quoi en faire.

JavaFX

`JavaFX` conserve ces quatre familles et y ajoute trois choses : la possibilité de décrire l'interface dans un fichier séparé plutôt qu'en code, une mise en forme par feuilles de style comparable au CSS du web, et la prise en charge des écrans tactiles.

C'est aujourd'hui la bibliothèque officielle. Le développement d'AWT et de Swing est arrêté — ils restent livrés avec le JDK, pour que les applications existantes continuent de fonctionner, mais ils n'évoluent plus.

> **AWT n'a pourtant pas disparu de votre écran**
>
> Swing est bâti sur AWT, et certaines classes AWT restent utilisées partout — notamment celles qui représentent les événements. Vous trouverez donc un `KeyEvent` dans `java.awt.event` et un autre dans `javafx.scene.input`.
>
> Les deux portent le même nom et n'ont rien à voir. Quand la complétion vous en propose un, lisez le nom complet avant de valider : importer celui d'AWT dans une application `JavaFX` donne un code qui ne compile pas, avec un message qui n'aide pas. Voir `IDE.11` du guide IntelliJ.

TEC-FX.2 — JavaFX ne fait plus partie du JDK

`JavaFX` a été livrée avec le JDK depuis Java 8, puis en a été retirée à partir de Java 11. Depuis, c'est un projet distinct, OpenJFX, développé en logiciel libre.

La conséquence est directe. Installer un JDK 21 ne vous donne pas `JavaFX`. Il faut l'ajouter au projet comme n'importe quelle bibliothèque externe, en la déclarant dans le `pom.xml` — voir `TEC-MVN.2`. C'est ce que fait le projet du cours, et c'est pourquoi le premier chargement télécharge davantage de choses qu'un projet ordinaire.

Cela explique aussi une bizarrerie que vous rencontrerez : une application `JavaFX` ne se lance pas toujours d'un simple clic sur la flèche verte, parce que la bibliothèque doit être signalée à la JVM au démarrage. Le projet du cours contient un greffon Maven qui s'en charge, et l'application se lance alors par la commande Maven correspondante.

Si vous voyez le message `JavaFX runtime components are missing`, c'est exactement ce problème : la bibliothèque est présente mais n'a pas été annoncée au lancement.

TEC-FX.3 — Les modules Java de JavaFX

`JavaFX` est découpée en sept modules. Un projet ne déclare que ceux dont il a besoin, ce qui évite d'embarquer un lecteur vidéo dans une application qui n'affiche que des boutons.

Module	Contenu
`javafx.base`	L'API de base : propriétés, liaisons, collections observables
`javafx.graphics`	Conteneurs, formes, images, animations, effets
`javafx.controls`	Les composants : boutons, champs, tableaux, listes
`javafx.fxml`	Le format FXML, qui décrit une interface en XML
`javafx.media`	Lecture audio et vidéo
`javafx.web`	Affichage de contenu web
`javafx.swing`	Passerelle pour intégrer du `JavaFX` dans une application Swing

Ce cours utilise les quatre premiers. `javafx.fxml` mérite une mention particulière : il permet de décrire la fenêtre dans un fichier `.fxml` séparé du code Java. La description de l'interface d'un côté, le comportement de l'autre — c'est ce qui rend praticable la séparation en modèle, vue et contrôleur.

TEC-FX.4 — Pourquoi une application JavaFX refuse de démarrer

Vous rencontrerez ce message, et il déroute parce qu'il semble faux : les bibliothèques sont bien là, Maven les a téléchargées, IntelliJ les affiche dans la ligne de commande.

```
Erreur : des composants d'execution JavaFX obligatoires pour
executer cette application sont manquants.
```

Ce qui se passe réellement

Une bibliothèque peut être fournie de deux façons. Sur le **classpath**, la manière historique, décrite en `TEC-CMP.3`. Ou sur le **chemin de modules**, apparu avec Java 9, où chaque bibliothèque déclare ce qu'elle offre et ce qu'elle exige.

Le lanceur du JDK examine la classe qu'on lui demande d'exécuter. **Si elle hérite d'Application**, il en déduit que c'est une application `JavaFX`, et il exige alors que `JavaFX` soit chargé comme module. Il vérifie, ne le trouve pas, et s'arrête — avant d'avoir exécuté la moindre ligne de votre code.

Or Maven, lui, place les bibliothèques sur le classpath ordinaire. Les deux ne s'entendent pas, et c'est tout le problème : *rien ne manque*, les jars sont présents et complets. C'est leur mode de chargement qui ne convient pas au contrôle.

La parade : une classe de lancement

Le contrôle ne se déclenche que si la classe principale hérite d'Application. Il suffit donc de lui en présenter une autre.

```
public class LanceurDemineur {

    public static void main(String[] args) {
        Application.launch(Demineur.class, args);
    }
}
```

Cette classe n'hérite de rien. Le lanceur ne reconnaît pas une application `JavaFX`, ne cherche pas de module, et exécute le `main`. C'est ensuite `Application.launch` qui démarre `JavaFX` — depuis le classpath, sans protester, parce que le contrôle est derrière nous.

> **Ce n'est pas un bricolage**
>
> C'est la forme recommandée dès qu'on distribue une application `JavaFX` sans passer par les modules, et vous la retrouverez telle quelle dans beaucoup de projets.
>
> Conséquence pratique : la classe à lancer n'est pas celle qui hérite d'Application. Dans ce cours, c'est toujours celle dont le nom commence par Lanceur.

Les deux autres solutions, et leur prix

Solution	Ce qu'elle coûte
`Ajouter --module-path et --add-modules aux options de la machine virtuelle`	le chemin dépend de la machine : la configuration d'exécution n'est plus transportable d'un poste à l'autre
`Déclarer un module-info.java`	la solution propre, mais elle impose que toutes les dépendances soient modulaires, tests compris
`Passer par mvn javafx:run`	le plugin monte le chemin de modules lui-même ; il faut penser à passer par la fenêtre Maven plutôt que par le bouton Run

La classe de lancement a l'avantage de ne rien demander : ni configuration, ni ligne de commande, ni fichier supplémentaire à tenir à jour.

De votre code au programme qui tourne

TEC-CMP.1 — Trois états de votre programme

Ce que vous écrivez et ce que la machine exécute ne sont pas la même chose. Entre les deux, il y a une traduction.

```
Personne.java          votre code, du texte que vous lisez
      |
      |   javac        le compilateur
      v
Personne.class         du bytecode, illisible pour vous
      |
      |   java         la machine virtuelle
      v
le programme s'execute
```

`javac` lit vos fichiers `.java` et produit des fichiers `.class`. C'est là que sont détectées les erreurs de compilation : une parenthèse manquante, un type qui ne correspond pas, une méthode qui n'existe pas.

`java` prend un fichier `.class` et l'exécute. C'est là qu'apparaissent les erreurs d'exécution : une division par zéro, un `null` inattendu, un fichier introuvable.

> **Pourquoi cette distinction compte**
>
> Elle vous dit dans quelle direction chercher. Une erreur signalée avant que le programme ne démarre vient de javac : le code est mal écrit. Une erreur qui interrompt un programme en cours de route vient de la JVM : le code est correct mais ce qu'il rencontre ne l'est pas.
>
> IntelliJ vous le dit d'ailleurs : les messages de compilation commencent par « java: », ceux d'exécution apparaissent dans la fenêtre Run avec une trace d'appels.

TEC-CMP.2 — Pourquoi Java tourne partout

Un programme écrit en C doit être recompilé pour chaque système. En Java, non : le même fichier .class s'exécute sur Windows, macOS et Linux.

```
        Personne.class
       /      |       \
     JVM     JVM      JVM
  Windows    Mac     Linux
```

Le bytecode n'est pas destiné à un processeur : il est destiné à la JVM. C'est elle qui diffère d'un système à l'autre, et qui traduit le bytecode en instructions que la machine comprend. Vous compilez une fois, vous exécutez partout — à condition qu'une JVM soit installée.

C'est ce qui explique aussi qu'un programme Java soit un peu plus lent à démarrer qu'un programme compilé directement : il y a une traduction supplémentaire au moment de l'exécution.

TEC-CMP.3 — Le classpath

Quand votre programme utilise `ArrayList`, la JVM doit trouver le fichier `ArrayList.class` quelque part. La liste des endroits où elle cherche s'appelle le **classpath**.

Il contient trois sortes de choses : la bibliothèque standard, fournie avec le JDK ; les bibliothèques externes, comme `JUnit` ; et vos propres classes compilées.

Vous n'aurez normalement jamais à le construire à la main : Maven le fabrique à partir des dépendances déclarées dans le `pom.xml`, et IntelliJ le transmet à la JVM au lancement.

Le connaître sert surtout à comprendre un message. `ClassNotFoundException` et `NoClassDefFoundError` veulent tous deux dire la même chose : une classe manque au classpath. Presque toujours, une dépendance n'a pas été téléchargée.

TEC-CMP.4 — Le dossier target

Maven range tout ce qu'il produit dans un dossier `target`, à la racine de chaque module. Vous y trouverez vos `.class`, les rapports de test, et l'archive `.jar` si vous en fabriquez une.

Rien de ce que contient ce dossier n'a de valeur. Tout s'y recrée intégralement à partir de `src` et du `pom.xml`. C'est pourquoi il est ignoré par Git, absent des archives que vous recevez, et pourquoi vous pouvez le supprimer sans crainte.

Le supprimer est d'ailleurs un réflexe utile quand quelque chose se comporte bizarrement : cela force une recompilation complète et élimine l'hypothèse d'un fichier compilé périmé.

Ce qui se passe pendant l'exécution

Pendant qu'il tourne, votre programme occupe deux zones de mémoire aux comportements très différents. Comprendre laquelle contient quoi éclaire une bonne partie de ce qui surprend en début d'apprentissage.

TEC-EXE.1 — La pile

Chaque fois qu'une méthode est appelée, la JVM empile un bloc contenant ses paramètres et ses variables locales. Quand la méthode se termine, le bloc est retiré et tout ce qu'il contenait disparaît.

D'où le nom : les blocs s'empilent et se dépilent, et on ne retire jamais que celui du dessus — c'est-à-dire la méthode en cours.

```
main()                      <-- appelle inscrire()
  inscrire(pseudos)         <-- appelle ajouter()
    ajouter(joueur)         <-- methode en cours d'execution

  ^ la pile : trois blocs, le dernier appele est en haut
```

Vous avez déjà vu cette pile sans le savoir : c'est exactement ce qu'affiche une trace d'exception, lue de bas en haut. Et le débogueur la montre en direct dans son panneau `Frames` — voir `IDE.19` du guide IntelliJ.

La pile a une taille limitée. Une méthode qui s'appelle elle-même sans condition d'arrêt empile des blocs jusqu'à la saturer, et la JVM lève alors une `StackOverflowError`. Ce message ne veut rien dire d'autre que : trop d'appels imbriqués, cherchez une récursion sans fin.

TEC-EXE.2 — Le tas

Les objets, eux, ne vivent pas dans la pile. Chaque `new` réserve de la place dans une seconde zone, le **tas**, et y construit l'objet.

La différence essentielle est de durée de vie. Le contenu de la pile disparaît dès que la méthode se termine ; un objet du tas survit tant que quelque chose le désigne. C'est ce qui permet à une méthode de créer un objet et de le retourner à son appelant — l'objet reste, seul le bloc de pile a disparu.

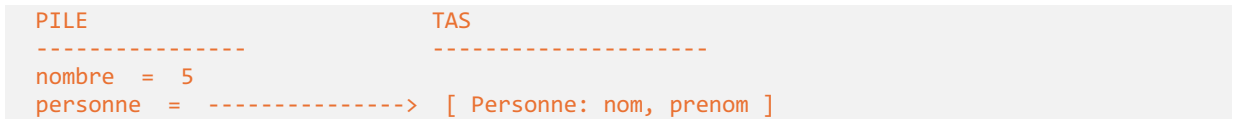
```
public Partie inscrire(String... pseudos) {
    Partie partie = new Partie();   // l'objet va dans le TAS
    ...                             // la variable est dans la PILE
    return partie;                  // le bloc de pile disparait,
}                                   // l'objet du tas survit
```

Le tas aussi a une limite. La saturer donne une `OutOfMemoryError` : trop d'objets créés, ou des objets qu'on n'arrive plus à libérer. Retenez la paire — `StackOverflowError` concerne les appels, `OutOfMemoryError` concerne les objets.

TEC-EXE.3 — Ce que contient vraiment une variable

Voici le point qui explique le plus de surprises de première année.

Une variable de type primitif — `int`, double, `boolean` — contient sa valeur. Une variable d'objet, non : elle contient une référence, c'est-à-dire de quoi retrouver l'objet dans le tas. L'objet est ailleurs.



Première conséquence : le passage de paramètres

Java passe toujours les arguments *par valeur* : la méthode reçoit une copie. Pour un `int`, la valeur copiée est le nombre, et modifier la copie ne change rien à l'original. Pour un objet, la valeur copiée est la *référence* — la copie désigne donc le même objet, et les modifications faites à travers elle sont visibles de partout.

Ce n'est pas un « passage par référence » : réaffecter le paramètre à l'intérieur de la méthode ne changerait rien chez l'appelant. C'est le comportement que l'exercice `POO.4` vous fait constater.

Seconde conséquence : deux noms, un objet

```
Personne a = new Personne();
Personne b = a;              // b designe le MEME objet

b.setNom("Lovelace");
System.out.println(a.getNom());   // affiche Lovelace
```

Le débogueur rend cela visible : il affiche un numéro à côté de chaque objet, `Personne@827` par exemple. Deux variables qui portent le même numéro désignent le même objet. Ce numéro n'a aucune signification en soi et change à chaque exécution — il ne sert qu'à distinguer les objets entre eux.

TEC-EXE.4 — Le ramasse-miettes

En Java, on ne libère jamais la mémoire à la main. Il n'existe pas de `delete`. Un programme séparé, le **ramasse-miettes** — garbage collector —, tourne en arrière-plan et récupère la place des objets que plus personne ne désigne.

```
Personne p = new Personne();   // l'objet est cree dans le tas
p = null;                      // plus rien ne le designe
                               // le ramasse-miettes le recuperera
                               // un jour, sans qu'on sache quand
```

Le critère est l'accessibilité, pas la portée : un objet reste vivant tant qu'une variable, un champ ou une collection le désigne, même indirectement. Un objet placé dans une liste survit à la disparition de la variable qui l'a créé.

Vous ne contrôlez pas le moment. Il existe bien une méthode `System.gc()`, mais elle ne fait que suggérer, et l'usage recommandé est de ne pas l'appeler. Écrivez votre code sans vous préoccuper de la libération : c'est précisément ce que Java vous offre par rapport à des langages où l'oubli d'un `free` est une fuite de mémoire.

À faire une fois, dans le débogueur

> **Voir la pile et le tas de ses propres yeux**
>
> Ouvrez la classe Inscription de l'exercice `PKG.4`. Posez un point d'arrêt sur la ligne qui ajoute un joueur à la partie, puis lancez Application en mode débogage.
>
> Au deuxième ou troisième arrêt, regardez le panneau Frames : vous y lisez la pile, inscrire appelée depuis main. Puis dépliez la variable partie dans le panneau Variables.
>
> Répondez alors à trois questions. Combien d'objets Joueur existent à cet instant ? Qui les désigne ? Et que deviendraient-ils si la méthode retournait autre chose que partie ?
>
> La marche à suivre dans l'outil est décrite en `IDE.19` du guide IntelliJ.

Maven

TEC-MVN.1 — À quoi sert Maven

Maven résout trois problèmes que vous n'avez pas encore rencontrés parce qu'il s'en occupe déjà.

- **Il impose une structure.** Le code dans `src/main/java`, les tests dans `src/test/java`, le résultat dans `target`. Cette convention est la même dans tous les projets Maven du monde, ce qui fait qu'on s'y retrouve immédiatement dans un projet inconnu.
- **Il gère les bibliothèques.** Vous déclarez que le projet a besoin de `JUnit` ; Maven va le chercher, ainsi que tout ce dont `JUnit` a lui-même besoin.
- **Il rend la compilation reproductible.** Le même projet compilé sur une autre machine donne le même résultat, avec les mêmes versions de tout.

Sans Maven, il faudrait télécharger les bibliothèques à la main, les ranger quelque part, et construire soi-même le classpath. C'est faisable, et c'est fastidieux.

TEC-MVN.2 — Le pom.xml

Tout tient dans un fichier, le `pom.xml` — Project `Object` Model. Il décrit le projet et ses besoins.

```
<groupId>be.ephec.java</groupId>       <!-- qui publie -->
<artifactId>exercices</artifactId>     <!-- quoi -->
<version>1.0-SNAPSHOT</version>        <!-- quelle version -->

<properties>
```

```
    <maven.compiler.release>21</maven.compiler.release>
</properties>

<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Les trois premières lignes forment les coordonnées du projet : elles l'identifient de façon unique, exactement comme elles identifient `JUnit` un peu plus bas. Toute bibliothèque du monde Java se désigne par ce triplet.

`maven.compiler.release` fixe la version de Java. C'est cette ligne qui garantit que votre code compilera de la même façon partout, quelle que soit la version du JDK installé — voir `TEC-PLA.3`.

`scope test` signifie que `JUnit` n'est disponible que pour le code de test. C'est volontaire : si vous tentez d'importer une classe `JUnit` dans `src/main/java`, cela ne compilera pas — et c'est tant mieux, du code de production n'a rien à faire avec une bibliothèque de test.

TEC-MVN.3 — Un projet, plusieurs modules Maven

Le projet du cours est composé de trois modules Maven, `exercices`, `solutions` et `demos`, chapeautés par un projet parent.

```
ExercicesJava/
    pom.xml            <-- le parent : declare les trois modules
    exercices/
        pom.xml        <-- herite du parent
        src/
    solutions/
        pom.xml
        src/
    demos/
        pom.xml
        src/
```

Le parent contient ce qui est commun : la version de Java, les versions des bibliothèques. Les modules n'ont plus qu'à déclarer ce qui leur est propre. Changer la version de Java se fait donc à un seul endroit.

Les trois modules sont indépendants les uns des autres : le module des solutions ne dépend pas de celui des exercices, ni celui des démonstrations d'aucun des deux, ce qui garantit qu'aucune classe de l'un ne peut être importée par un autre par accident. Une erreur de compilation dans vos exercices n'empêche donc pas de lancer une démonstration.

TEC-MVN.4 — Le cycle de vie

Maven définit une suite d'étapes ordonnées. Demander une étape exécute automatiquement toutes celles qui la précèdent.

Étape	Ce qu'elle fait
`compile`	Compile src/main/java vers target/classes
`test`	Compile les tests et les exécute (compile d'abord)
`package`	Fabrique le .jar (compile et test d'abord)
`clean`	Supprime target — s'utilise seule, hors du cycle

Vous n'aurez pas souvent à les lancer vous-même : IntelliJ le fait pour vous quand vous cliquez sur la flèche verte. Elles apparaissent dans la fenêtre Maven, sous `Lifecycle`, et se lancent d'un double clic.

Le seul cas où vous y viendrez : `clean` puis un nouveau lancement, quand vous soupçonnez un fichier compilé périmé. Voir `TEC-CMP.4`.

TEC-MVN.5 — Le dépôt local

Les bibliothèques téléchargées ne sont pas rangées dans le projet, mais dans un dossier commun à tous vos projets : `.m2/repository`, dans votre dossier personnel.

C'est ce qui explique la minute d'attente au premier chargement, et son absence les fois suivantes : Maven regarde d'abord dans ce dépôt local, et ne va sur Internet que si la bibliothèque y manque. Un second projet utilisant `JUnit` 5.10.2 la trouvera déjà là.

Ce dossier peut être supprimé sans risque : tout se retéléchargera. C'est le remède de dernier recours quand une bibliothèque semble corrompue — mais comptez alors à nouveau le temps de téléchargement.

Recevoir les fichiers du cours

TEC-ZIP.1 — Extraire une archive au bon endroit

Chaque séance donne lieu à une archive. Elle contient elle-même un dossier `ExercicesJava`, au même nom que le vôtre :

```
SeanceNN_..._exercices.zip
    ExercicesJava/
        exercices/src/main/java/be/ephec/java/tab/...
        exercices/src/test/java/be/ephec/java/tab/...
        enonces/TAB_enonces.docx
```

Elle doit donc être extraite dans le dossier PARENT du vôtre, pas à sa racine. Les deux arborescences fusionnent alors, et les nouveaux packages viennent s'ajouter aux anciens. Extraite à la racine, elle produirait un absurde `ExercicesJava/ExercicesJava/...` que Maven ignorerait.

Pour retrouver ce dossier parent sans se tromper : dans IntelliJ, clic droit sur la racine du projet, `Open In` puis `Finder` ou `Explorer`, et remontez d'un niveau.

macOS : passez par le Terminal

Le double-clic ne convient pas. L'Utilitaire d'archive extrait toujours à côté du fichier zip, et s'il rencontre un dossier du même nom il en crée un second — « ExercicesJava 2 » — au lieu de fusionner. Vous vous retrouvez avec deux projets, dont un incomplet.

```
unzip -o ~/Downloads/SeanceNN_..._exercices.zip -d ~/chemin/du/parent
```

`unzip` fusionne dans les dossiers existants, ce que le Finder ne fait pas de façon fiable. L'option `-o` évite qu'il pose une question pour chaque fichier.

Astuce : tapez `unzip -o ` puis faites glisser le fichier zip depuis le Finder dans la fenêtre du Terminal — son chemin s'écrit tout seul. Même chose pour le dossier de destination après `-d`.

> **Pourquoi pas le glisser-déposer**
>
> On serait tenté d'extraire n'importe où puis de faire glisser le dossier ExercicesJava sur celui du projet. Le Finder propose alors « Remplacer », ce qui supprimerait tout votre travail, et ne propose « Fusionner » que dans certains cas.
>
> Le risque n'en vaut pas la peine : une ligne de Terminal fait le travail sans ambiguïté.

Windows : Extraire tout, en corrigeant le chemin

Clic droit sur le zip, `Extraire tout`. Une boîte de dialogue demande la destination, et Windows fusionne correctement les dossiers.

Attention au chemin proposé. Windows ajoute par défaut un sous-dossier portant le nom du zip. Effacez cette partie pour ne garder que le dossier parent de votre projet, sinon vous obtiendrez `...\SeanceNN_..._exercices\ExercicesJava\...` — un niveau de trop.

En PowerShell, si vous préférez :

```
Expand-Archive -Path "$HOME\Downloads\SeanceNN_..._exercices.zip" `
               -DestinationPath "C:\chemin\parent" -Force
```

TEC-ZIP.2 — Ne pas écraser son propre travail

L'archive d'une séance n'apporte que les packages de cette séance-là. Elle ne touche donc pas à ce que vous avez écrit lors des séances précédentes, et il n'y a aucun risque à l'extraire.

En revanche, extraire deux fois l'archive d'une même séance écraserait votre travail sur ces exercices-là. Les squelettes reviendraient à leur état d'origine, avec leurs `UnsupportedOperationException`, et votre code aurait disparu.

Si vous devez reprendre une archive déjà extraite — parce que vous avez cassé quelque chose, ou pour comparer —, extrayez-la dans un dossier temporaire et recopiez seulement le fichier dont vous avez besoin.

C'est exactement le genre d'accident qu'un gestionnaire de versions rend impossible, puisqu'il conserve l'état de chaque fichier à chaque instant. Voir `TEC-GIT.1`.

Git, en deux mots

Git n'est au programme d'aucun exercice, et il ne sera pas évalué. Vous n'aurez jamais à enregistrer vos modifications, ni à travailler à plusieurs, ni à envoyer quoi que ce soit sur un serveur.

Une exception, et en lecture seule. Le corrigé du fil rouge GameHub peut vous être remis sous la forme d'un dépôt Git, qui conserve les trente incréments et permet de se placer à n'importe lequel. Quatre gestes suffisent à le parcourir, et ils sont décrits en `TEC-GIT.3`.

Les deux premières sections existent pour que le mot ne soit pas un mystère, et pour expliquer certains fichiers que vous verrez dans le projet.

TEC-GIT.1 — Ce qu'est un gestionnaire de versions

Un gestionnaire de versions enregistre l'historique d'un projet. À intervalles choisis, on lui demande de mémoriser l'état complet des fichiers — cela s'appelle un commit — avec un message décrivant ce qui a changé.

Il devient alors possible de revenir à n'importe quel état antérieur, de voir ce qui a changé entre deux moments, et de savoir qui a modifié quoi. À plusieurs, chacun travaille de son côté et le système fusionne les contributions.

Git est le plus répandu d'entre eux, et de très loin. GitHub, GitLab et Bitbucket sont des services qui hébergent des dépôts Git — ce ne sont pas des outils différents, mais des endroits où poser un dépôt.

La conséquence pratique pour vous. Puisque l'historique est conservé, on ne garde jamais de code en commentaire « au cas où ». C'est le fondement de la règle `CC-COM.2` du mémento Clean Code.

TEC-GIT.2 — Pourquoi certains dossiers sont ignorés

Le projet contient un fichier `.gitignore` listant ce que Git doit laisser de côté. Vous y trouverez au moins ceci :

```
target/      # tout se recree a la compilation
.idea/       # preferences d'affichage d'IntelliJ, propres a chaque poste
*.iml
```

La règle qui les réunit : on ne conserve dans l'historique que ce qu'on ne peut pas refabriquer. Un fichier compilé se refabrique à partir de la source. Une préférence d'affichage n'a d'intérêt que pour la personne qui l'a réglée, et l'inclure ferait entrer en conflit les réglages de chacun.

C'est aussi pourquoi vous pouvez supprimer `.idea` sans rien perdre d'important quand IntelliJ se comporte étrangement — remède mentionné en `ERR.1` du guide IntelliJ.

TEC-GIT.3 — Le dépôt du fil rouge, et comment le parcourir

Le corrigé de GameHub vous est remis sous la forme d'un **dépôt Git**. Ce n'est pas un fichier particulier : c'est un dossier ordinaire, qui contient en plus un sous-dossier caché `.git` où vit l'historique des trente incréments.

Vous l'ouvrez dans IntelliJ comme n'importe quel projet, et vous pouvez alors vous placer à n'importe quelle étape du fil rouge.

Trois notions, et rien d'autre

Terme	Ce que c'est
`commit`	l'état complet du projet à un instant, sous un nom lisible
`historique`	la suite des commits, chacun ne retenant que ce qui a changé
`étiquette (tag)`	un nom collé sur un commit — ici G.1 à G.30

Ce que le dépôt apporte et qu'un document ne peut pas donner : la comparaison. Choisissez un incrément, et IntelliJ affiche les deux versions d'un fichier côte à côte, en marquant en vert ce qui a été ajouté et en bleu ce qui a été modifié.

> **Le mode d'emploi est ailleurs, et c'est voulu**
>
> Git est un outil vaste, et il n'est pas au programme. Vous l'utiliserez en lecture seule, et quatre gestes suffisent.
>
> Ils sont décrits pas à pas, captures d'écran à l'appui, dans `Git_pour_GameHub.docx` — livré avec le dépôt, dans son dossier _documents. Ce mémento ne les recopie pas : deux documents qui disent la même chose finissent toujours par se contredire.

Deux choses à savoir avant d'ouvrir le dépôt

- **Git doit être installé sur la machine.** IntelliJ ne l'embarque pas. Sans lui, l'onglet Git n'apparaît pas et rien de ce qui précède ne fonctionne.
- **Déplacez toujours le dossier entier.** Le `.git` est caché : copier seulement les sources fait disparaître les trente versions sans qu'aucun message ne le signale.

Et un conseil qui évite la plupart des ennuis : **n'écrivez pas dans le dépôt**. C'est un corrigé qui se lit. Pour essayer des choses, copiez le code dans un projet à vous.

Annexe C — Mémento Clean Code

Des règles simples pour écrire du code lisible

À quoi sert ce mémento

Un programme est lu bien plus souvent qu'il n'est écrit — par d'autres, et par vous-même dans six mois. Les règles rassemblées ici ne rendent pas votre code plus rapide ni plus correct : elles le rendent lisible. C'est ce qui décide, en pratique, si un programme peut encore être corrigé un an plus tard.

Elles viennent pour l'essentiel de *Clean Code: A Handbook of Agile Software Craftsmanship*, de Robert C. Martin. Elles ne sont pas des lois : ce sont des réflexes par défaut, dont on s'écarte parfois en sachant pourquoi.

Comment l'utiliser

Ne le lisez pas d'une traite. Chaque règle porte un code, et les énoncés d'exercices y renvoient au moment où elle devient utile : les règles de nommage dès la première séance, celles sur les erreurs avec le module `EXC`, celles sur les tests avec JUN.

La table qui suit sert de sommaire : cliquez sur une règle pour aller à son explication. Les renvois d'une règle à l'autre dans le texte sont eux aussi cliquables.

Le code se lit ainsi : `CC-ERR.2` désigne la deuxième règle de la section « Traiter les erreurs ». Le préfixe encode le sujet, pas le rang : l'ajout d'une règle ne renumérote jamais les autres.

Ces règles comptent à l'examen. Le critère de style de la grille d'évaluation s'appuie sur elles, et un commentaire de correction peut y renvoyer directement.

Les règles en un coup d'œil

Nommer	
`CC-NOM.1`	Le nom communique l'intention
`CC-NOM.2`	Le nom ne dit pas autre chose que ce qu'il désigne
`CC-NOM.3`	Le nom se prononce
`CC-NOM.4`	Le nom n'encode pas le type
`CC-NOM.5`	Noms pour les choses, verbes pour les actions, prédicats pour les booléens
`CC-NOM.6`	La longueur du nom suit la portée
`CC-NOM.7`	Pas de nombre magique : nommez-le
`CC-NOM.8`	Déclarez une variable près de l'endroit où elle sert

Écrire une méthode	
CC-MET.1	Le nom de la méthode dit ce qu'elle fait
CC-MET.2	Une méthode reste courte
CC-MET.3	Une méthode ne fait qu'une seule chose
CC-MET.4	Le vocabulaire est cohérent d'une méthode à l'autre
CC-MET.5	Pas de drapeau booléen en paramètre
CC-MET.6	Deux paramètres au plus
CC-MET.7	Pas d'effet de bord
CC-MET.8	Une méthode agit ou répond, jamais les deux
CC-MET.9	Un booléen se retourne tel quel

Commenter	
CC-COM.1	Un commentaire nécessaire signale un code à réécrire
CC-COM.2	Pas de code laissé en commentaire
CC-COM.3	Une Javadoc ne répète pas la signature

Objets et encapsulation	
CC-OBJ.1	Demandez à l'objet d'agir, ne lui demandez pas ses entrailles
CC-OBJ.2	Un accesseur n'est pas une obligation

Traiter les erreurs	
CC-ERR.1	Une erreur se signale par une exception, pas par une valeur de retour
CC-ERR.2	Ne retournez pas null
CC-ERR.3	Ne passez pas null en paramètre
CC-ERR.4	Le message d'exception dit ce qui a échoué et avec quelle valeur
CC-ERR.5	Définissez vos exceptions selon ce que l'appelant doit distinguer
CC-ERR.6	N'avalez pas une exception en silence
CC-ERR.7	Une comparaison se protège de null

Écrire des tests	
CC-TST.1	Le code de test se soigne comme le code de production
CC-TST.2	Un test vérifie une seule chose
CC-TST.3	Un test est rapide, indépendant, répétable et auto-vérifiant
CC-TST.4	Un test se termine par une assertion

Structurer une classe	
CC-CLS.1	Une classe n'a qu'une raison de changer
CC-CLS.2	Une classe reste petite
CC-CLS.3	Ne vous répétez pas
CC-CLS.4	Préférez le polymorphisme à une chaîne de tests sur un type

Nommer

Les noms que vous choisissez sont un moyen de communication : avec les autres développeurs, avec le correcteur, et avec vous-même dans trois ans. C'est la seule partie du code que personne ne peut deviner à votre place.

CC-NOM.1 — Le nom communique l'intention

Si vous avez besoin d'un commentaire pour expliquer ce que contient une variable, c'est que son nom ne le dit pas. Mettez l'explication dans le nom et supprimez le commentaire.

À ÉVITER
```
int d;   // temps ecoule en jours
```

PRÉFÉREZ
```
int tempsEcouleEnJours;
```

L'effet se voit surtout sur un calcul un peu long. Voici la méthode que Robert C. Martin donne en exemple, avant et après :

À ÉVITER
```
public static Date addMonths(int months, Date base) {
    int yy = (12 * base.getYear() + base.getMonth() + months - 1) / 12;
    int mm = (12 * base.getYear() + base.getMonth() + months - 1) % 12 + 1;
    int dd = Math.min(base.getDayOfMonth(), Date.lastDayOfMonth(mm, yy));
    return Date.createInstance(dd, mm, yy);
}
```

PRÉFÉREZ
```
public static Date addMonths(int monthsToAdd, Date base) {
    int baseYear         = base.getYear();
    int baseMonth        = base.getMonth() - JANUARY;
    int monthsInBaseYear = 12 * baseYear;
    int baseMonths       = baseMonth + monthsInBaseYear;
    int resultMonths     = baseMonths + monthsToAdd;
    int resultYear       = resultMonths / 12;
    int resultMonth      = resultMonths % 12 + JANUARY;
    int lastDay          = Date.lastDayOfMonth(resultMonth, resultYear);
    int resultDay        = Math.min(base.getDayOfMonth(), lastDay);
    return Date.createInstance(resultDay, resultMonth, resultYear);
}
```

La seconde version est plus longue et fait exactement la même chose. Mais on peut la lire : les noms racontent le calcul, là où la première demandait de le reconstituer.

CC-NOM.2 — Le nom ne dit pas autre chose que ce qu'il désigne

Un nom qui ment est pire qu'un nom vague : il installe une certitude fausse. Veillez à ce que le mot signifie ce qu'il indique, et qu'il indique ce qu'il signifie.

À ÉVITER
```
public class Pair {
    private Object first, second, third;

    public Pair(Object first, Object second, Object third) { ... }
}
```

PRÉFÉREZ
```
public class Triplet {
    private Object first, second, third;

    public Triplet(Object first, Object second, Object third) { ... }
}
```

Ce cas arrive rarement d'emblée : il naît d'une évolution. La classe était bien une paire, on lui a ajouté un troisième élément, et personne n'a renommé. Quand le code change, les noms doivent suivre — `Maj+F6` dans IntelliJ renomme partout sans rien casser.

Même piège avec un nom trop précis : appeler `listeEtudiants` une variable de type `Set` induit en erreur. Préférez etudiants.

CC-NOM.3 — Le nom se prononce

Le test est simple : imaginez que vous devez en parler à voix haute à un collègue, au téléphone. Un nom qu'on doit épeler est un nom à changer.

À ÉVITER
```
private int pq_gwta;

private int getYYYY() {
    return this.year;
}
```

PRÉFÉREZ
```
private int pointsGagnesAuTotal;

private int getAnnee() {
    return this.annee;
}
```

Ce n'est pas un détail de confort : un nom qu'on ne peut pas prononcer ne peut pas être discuté, et un nom qui ne peut pas être discuté ne sera jamais corrigé.

CC-NOM.4 — Le nom n'encode pas le type

La notation hongroise, inventée en 1991, préfixait chaque variable par une abréviation de son type. Elle avait un sens à l'époque des éditeurs de texte sans coloration ni saut à la déclaration. Aujourd'hui votre IDE vous donne le type en survolant l'identifiant, et le préfixe ne fait qu'ajouter du bruit — sans compter qu'il devient faux dès que le type change.

À ÉVITER
```
private boolean bDanger;
private int     iValeur;
private String  strNom;
```

PRÉFÉREZ
```
private boolean enDanger;
private int     valeur;
private String  nom;
```

Une exception tolérée : le préfixe `I` devant un nom d'interface (`IForme`). C'est une habitude venue du monde .NET, encore répandue, et elle a le mérite de distinguer l'interface de son implémentation. Vous la rencontrerez ; en Java, la convention majoritaire est de ne pas l'employer.

CC-NOM.5 — Noms pour les choses, verbes pour les actions, prédicats pour les booléens

Un code bien nommé se lit comme une phrase. Cela suppose de respecter la nature grammaticale de ce qu'on nomme.

Ce qu'on nomme	
`classe`	un nom : Compte, `CompteEpargne`, `MessageParser`, `DocumentArchiver`
`variable`	un nom : solde, `joueurCourant`, `listeDeCartes`
`méthode`	un verbe : regrouper, imprimer, envoyer, identifier, creer
`booléen`	un prédicat, qui se lit après un `if` : `estActif`, `isClosed`, `estVide`
`enum`	souvent un adjectif ou un état : TODO, BUSY, DONE, SMALL, LARGE

Le résultat se lit à voix haute sans effort :

PRÉFÉREZ
```
if (panier.estVide())
    afficherMessageDeBienvenue();

if (paiement.estPossible())
    enregistrerPaiement(paiement);

if (etudiant.getPoints(cours).estCodeEchec())
```

```
    etudiant.recommence(cours);
```

CC-NOM.6 — La longueur du nom suit la portée

Pour une variable, plus la portée est large, plus le nom doit être explicite. Un compteur de boucle utilisé sur trois lignes peut s'appeler i ; une variable déclarée en début de méthode et relue trente lignes plus bas ne le peut pas.

À ÉVITER
```
Tracker tr = new Tracker();

// ... trente lignes de code ...

for (int i = 0; i < 10; i++) {
    Message m = new Message();
    m.setSujet(tr.getNom());
    m.envoyer();
}
```

PRÉFÉREZ
```
Tracker tracker = new Tracker();

// ... trente lignes de code ...

for (int i = 0; i < 10; i++) {
    Message message = new Message();
    message.setSujet(tracker.getNom());
    message.envoyer();
}
```

Pour les méthodes et les classes, la règle s'inverse. Ce qui est public et largement utilisé porte un nom court ; ce qui est privé et sert à un seul endroit porte un nom long et descriptif.

PRÉFÉREZ
```
public void agir() {
    tournerDUnAngleAleatoire();
    chercherUnVerDeTerre();
}
```

CC-NOM.7 — Pas de nombre magique : nommez-le

Un nombre écrit en dur au milieu du code ne dit ni d'où il vient ni ce qu'il représente. Le lecteur doit le deviner, et celui qui devra le changer devra d'abord tous les retrouver.

À ÉVITER
```
if (joueur.getPoints() > 10) {
    joueur.promouvoir();
}

double total = montant * 1.21;
```

PRÉFÉREZ
```
private static final int    POINTS_POUR_PROMOTION = 10;
private static final double TAUX_TVA_STANDARD     = 1.21;

if (joueur.getPoints() > POINTS_POUR_PROMOTION) {
    joueur.promouvoir();
}

double total = montant * TAUX_TVA_STANDARD;
```

La constante fait deux choses à la fois : elle explique le nombre, et elle le rassemble en un seul endroit. Le jour où le taux de TVA change, il y a une ligne à modifier, pas une chasse à mener dans tout le projet.

Tous les nombres ne sont pas magiques. Le critère n'est pas la valeur, c'est ce qu'elle encode. Un nombre qui traduit une décision — un seuil, un taux, une limite — pourrait changer un jour, et il mérite un nom. Un nombre qui appartient à la mécanique du code ne changera jamais et se passe très bien du sien.

À ÉVITER
```
for (int i = 0; i < joueurs.size(); i++) {
    ...
}

if (joueurs.size() > 4) {
    ...
}
```

PRÉFÉREZ
```
// la boucle ne change pas : le 0 et le i++ sont la mecanique du parcours
for (int i = 0; i < joueurs.size(); i++) {
    ...
}

// seul le 4 disparait : lui seul encodait une decision
if (joueurs.size() > SEUIL_GRAND_GROUPE) {
    ...
}
```

Comparez les deux blocs : la boucle est identique. Ce n'est pas un oubli — c'est la démonstration que ses nombres n'avaient rien à se reprocher. Un seul des trois méritait un nom.

Le test à s'appliquer : si quelqu'un vous demandait pourquoi ce nombre-là plutôt qu'un autre, auriez-vous quelque chose à répondre ? Si oui, la réponse est le nom de la constante.

Le problème n'est d'ailleurs pas propre aux nombres. Une chaîne de caractères recopiée à plusieurs endroits pose exactement la même difficulté, et une faute de frappe y passe inaperçue — le compilateur ne vérifie pas le contenu d'un littéral.

À ÉVITER
```
if (utilisateur.getRole().equals("ADMIN")) { ... }

if (autre.getRole().equals("ADMLN")) { ... }      // qui verra la faute ?
```

PRÉFÉREZ
```
private static final String ROLE_ADMIN = "ADMIN";

if (utilisateur.getRole().equals(ROLE_ADMIN)) { ... }

if (autre.getRole().equals(ROLE_ADMLN)) { ... }   // ne compile pas
```

CC-NOM.8 — Déclarez une variable près de l'endroit où elle sert

Grouper toutes les déclarations en tête de méthode est une habitude héritée de langages qui l'imposaient. En Java, elle oblige le lecteur à faire des allers-retours et élargit inutilement la portée de chaque variable.

À ÉVITER
```
public void distribuer(List<Joueur> joueurs) {
    Carte carte;
    int i;
    Joueur joueur;

    Collections.shuffle(paquet);

    for (i = 0; i < joueurs.size(); i++) {
        joueur = joueurs.get(i);
        carte = paquet.remove(0);
        joueur.recevoir(carte);
    }
}
```

PRÉFÉREZ
```
public void distribuer(List<Joueur> joueurs) {
    Collections.shuffle(paquet);

    for (Joueur joueur : joueurs) {
        Carte carte = paquet.remove(0);
        joueur.recevoir(carte);
```

```
    }
}
```

Chaque variable naît là où elle sert et disparaît avec le bloc. C'est CC-NOM.6 vu sous un autre angle : plutôt que d'allonger le nom pour compenser une portée large, réduisez la portée.

Écrire une méthode

Une méthode est l'unité de lecture d'un programme. Si elle tient dans un écran et porte un nom exact, on peut la comprendre sans lire ce qu'elle appelle — et c'est ce qui rend un programme entier abordable.

CC-MET.1 — Le nom de la méthode dit ce qu'elle fait

Le nom doit suffire à l'appelant. S'il faut ouvrir le corps de la méthode pour savoir ce qu'elle fait exactement, le nom est incomplet.

À ÉVITER
```
public LocalDate addToDate(LocalDate date, int month) {
    return date.plus(month, ChronoUnit.MONTHS);
}

LocalDate date = LocalDate.now();
date = addToDate(date, 3);      // trois quoi ? jours ? mois ?
```

PRÉFÉREZ
```
public LocalDate addMonthsToDate(LocalDate date, int months) {
    return date.plus(months, ChronoUnit.MONTHS);
}

LocalDate date = LocalDate.now();
date = addMonthsToDate(date, 3);
```

Notez au passage que le paramètre est passé au pluriel : `months` et non `month`, puisqu'il en désigne plusieurs. C'est CC-NOM.1 appliqué à un paramètre.

CC-MET.2 — Une méthode reste courte

Deux à cinq lignes est l'objectif. C'est court, et volontairement : le but n'est pas d'écrire moins de code, mais de le répartir dans des méthodes qui portent chacune un nom.

Le moyen d'y arriver n'est pas de compter les lignes, c'est de limiter l'indentation. Le corps d'une méthode devrait tenir en une conditionnelle, une boucle ou un `try/catch`, contenant un seul appel.

À ÉVITER
```
public void afficherElements(List<String> liste) {
    if (liste.isEmpty()) {
        System.out.println("La liste est vide.");
    } else {
        for (String element : liste) {
            if (!element.isEmpty()) {
                System.out.println("- " + element);
            }
        }
    }
}
```

PRÉFÉREZ
```
public void afficherElements(List<String> liste) {
    if (liste.isEmpty()) {
        System.out.println("La liste est vide.");
    } else {
        afficherElementsNonVides(liste);
    }
}

private void afficherElementsNonVides(List<String> liste) {
    for (String element : liste) {
        if (!element.isEmpty()) {
```

```
            System.out.println("- " + element);
        }
    }
}
```

Il y a maintenant deux méthodes au lieu d'une, et plus de lignes au total. Ce n'est pas un échec : chacune se lit d'un coup d'œil, et `afficherElementsNonVides` porte un nom qui explique ce que faisait la boucle imbriquée.

CC-MET.3 — Une méthode ne fait qu'une seule chose

Le test pratique : si vous décrivez ce que fait la méthode et que votre phrase contient un « et », ou descend dans le détail de l'implémentation, elle en fait plusieurs.

À ÉVITER
```
public static void emailClients(List<Client> clients) {
    for (Client client : clients) {
        ClientRecord record = database.lookup(client);
        if (record.isActive()) {
            sendEmail(client);
        }
    }
}
```

PRÉFÉREZ
```
public static void emailClients(List<Client> clients) {
    for (Client client : clients) {
        if (isActiveClient(client)) {
            sendEmail(client);
        }
    }
}

private static boolean isActiveClient(Client client) {
    ClientRecord record = database.lookup(client);
    return record.isActive();
}
```

La première version mélange deux niveaux : l'intention (envoyer un courriel aux clients actifs) et le moyen de savoir si un client est actif (une consultation en base). La seconde les sépare. Effet secondaire agréable, `isActiveClient` devient réutilisable ailleurs.

CC-MET.4 — Le vocabulaire est cohérent d'une méthode à l'autre

Des actions semblables portent le même verbe. Sinon le lecteur cherche une différence de sens là où il n'y en a pas.

À ÉVITER
```
public void loadSingleData();
public void fetchDataFiltered();
public void getAllData();
```

PRÉFÉREZ
```
public void getSingleData();
public void getFilteredData();
public void getAllData();
```

Quelle est la différence entre load, fetch et get ? Aucune, dans cet exemple — et c'est précisément le problème : le lecteur suppose qu'il y en a une. Choisissez un verbe par famille d'actions et tenez-vous-y, dans une classe comme dans une équipe.

CC-MET.5 — Pas de drapeau booléen en paramètre

Un booléen qui pilote le comportement d'une méthode annonce qu'elle fait deux choses. Il rend aussi les appels illisibles : rien ne dit ce que signifie `true`.

À ÉVITER
```
double d = distance(pointA, pointB, true);
```

PRÉFÉREZ

```
double d = distanceEnKilometres(pointA, pointB);
double d = distanceEnMiles(pointA, pointB);
```

Deux méthodes au lieu d'une, chacune avec un nom qui dit ce qu'elle retourne. C'est CC-MET.3 vu depuis la signature : un drapeau booléen est presque toujours le symptôme d'une méthode qui n'a pas été coupée en deux.

CC-MET.6 — Deux paramètres au plus

Chaque paramètre supplémentaire est un ordre à retenir et une occasion de se tromper. Au-delà de deux, c'est en général que plusieurs d'entre eux forment un ensemble qui mérite sa propre classe.

À ÉVITER
```
public void setCustomerAddress(String firstName, String lastName,
                               String streetAddress, String city,
                               String zipCode, String state,
                               String country, String phoneNumber)
```

PRÉFÉREZ
```
public void setCustomerAddress(Address address)
```

Le regroupement n'est pas qu'un artifice de comptage : une adresse est bel et bien une chose, et lui donner une classe permet de la valider, de la comparer et de la transporter d'un bloc.

> **Deux précisions**
>
> Un paramètre variadique en dernière position (`String... valeurs`) compte pour un seul argument.
>
> Un record est souvent le moyen le plus léger de regrouper des paramètres liés — vous le verrez au module ENU.

CC-MET.7 — Pas d'effet de bord

Un effet de bord, c'est ce que la méthode fait en plus de ce que son nom annonce. Personne ne s'y attend, donc personne ne s'en méfie.

Premier cas, l'effet caché sur les données. Imaginez ces deux méthodes :

À ÉVITER
```
public Product getProductById(List<Product> products, int id) { ... }

public int getProductIndexInList(List<Product> products, Product product) { ... }
```

Si l'auteur de `getProductById` décide de trier la liste pour accélérer sa recherche, tous les indices retournés auparavant par `getProductIndexInList` deviennent faux. Le nom de la méthode ne laissait rien deviner de ce tri. Une méthode dont le nom commence par `get` ne doit rien modifier.

Second cas, le mélange du traitement et de l'affichage. C'est de loin le plus fréquent dans vos exercices :

À ÉVITER
```
public class CompteBancaire {
    private double solde;

    public void retirerArgent(double montant) {
        if (solde >= montant) {
            solde -= montant;
            System.out.println("Retrait de " + montant + " effectue.");
        } else {
            System.out.println("Fonds insuffisants.");
        }
    }
}
```

La méthode retire de l'argent, ce qui est son travail, mais elle décide aussi de ce qui s'affiche à l'écran — une décision qui ne lui appartient pas. Elle devient inutilisable dans une application graphique, et intestable : il n'y a rien à vérifier qu'un texte parti sur la console.

PRÉFÉREZ
```
public class CompteBancaire {
    private double solde;
```

```
    public boolean retirerArgent(double montant) {
        if (solde < montant) {
            return false;
        }
        solde -= montant;
        return true;
    }
}

// la couche de presentation decide de ce qui s'affiche
if (compte.retirerArgent(100)) {
    System.out.println("Retrait de 100 effectue.");
} else {
    System.out.println("Fonds insuffisants.");
}
```

Une variante à ne pas retenir. On voit parfois la méthode rester `void` et la classe exposer un `dernierRetraitOK()` que l'appelant interroge après coup. Cela corrige bien l'affichage, mais introduit un défaut plus grave : la classe mémorise le résultat de la dernière opération, donc deux appels successifs se marchent dessus et l'ordre des instructions devient porteur de sens. Préférez la valeur de retour — ou l'exception, voir CC-ERR.1.

CC-MET.8 — Une méthode agit ou répond, jamais les deux

Une méthode modifie l'état de l'objet — c'est une commande, et elle ne retourne rien — ou bien elle répond à une question sans rien modifier — c'est une requête, et elle retourne une valeur. Mélanger les deux produit des appels dont on ne sait pas lire l'intention.

À ÉVITER
```
public boolean setAttribut(String cle, String valeur) { ... }

if (setAttribut("nom", "Lovelace")) {
    ...
}
```

Que teste ce `if` ? Que l'attribut existait déjà ? Que l'écriture a réussi ? Impossible de le savoir sans ouvrir la méthode. Séparez :

PRÉFÉREZ
```
public void    setAttribut(String cle, String valeur) { ... }   // commande
public boolean contientAttribut(String cle)           { ... }   // requete

if (contientAttribut("nom")) {
    setAttribut("nom", "Lovelace");
}
```

Cette règle éclaire la nuance de CC-ERR.1 : `estDecouvert()` est une requête, elle retourne légitimement un booléen ; `retirer()` est une commande, donc son échec se signale par une exception et non par une valeur de retour.

Un repère de nommage utile : une commande porte un verbe d'action (`ajouter`, `retirer`, `demarrer`), une requête porte un nom ou un prédicat (`solde`, `estVide`). Voir CC-NOM.5.

CC-MET.9 — Un booléen se retourne tel quel

Une comparaison comme `a > b` vaut déjà `true` ou `false` : c'est un booléen. Le reconstruire avec un `if` n'ajoute que des lignes.

À ÉVITER
```
public boolean estMajeur() {
    if (age >= 18) {
        return true;
    } else {
        return false;
    }
}
```

Le ternaire ne fait pas mieux : `return age >= 18 ? true : false;` est la même redondance, en une ligne. La comparaison se retourne directement :

PRÉFÉREZ
```
public boolean estMajeur() {
    return age >= 18;
}
```

La même règle vaut pour les comparaisons avec `true` : `if (estMajeur() == true)` s'écrit `if (estMajeur())`, et `if (estMajeur() == false)` s'écrit `if (!estMajeur())`. Un prédicat bien nommé (CC-NOM.5) se lit alors comme une phrase.

Commenter

Un commentaire n'est ni compilé, ni testé, ni mis à jour par personne. Il vieillit donc plus vite que le code qu'il décrit, et un commentaire faux fait plus de dégâts qu'un commentaire absent.

CC-COM.1 — Un commentaire nécessaire signale un code à réécrire

Avant d'écrire un commentaire, demandez-vous s'il n'existe pas un moyen de le dire dans le code. La plupart du temps, il suffit d'extraire une méthode et de lui donner le nom que vous alliez écrire en commentaire.

À ÉVITER
```
// verifie si l'employe a droit a tous les avantages
if (employe.getAnciennete() > 5 && employe.getStatut() == Statut.TEMPS_PLEIN) {
    ...
}
```

PRÉFÉREZ
```
if (employe.aDroitATousLesAvantages()) {
    ...
}
```

Le commentaire a disparu et l'information est toujours là — mieux, elle est désormais réutilisable et testable. C'est le même mouvement qu'en CC-NOM.1, appliqué cette fois à une expression.

Restent les commentaires qui ont leur place : expliquer un pourquoi qu'aucun nom ne peut porter — une contrainte légale, un contournement de bug, une décision de conception surprenante. Ceux-là, écrivez-les.

PRÉFÉREZ
```
// L'arrondi se fait a 2 decimales : impose par la reglementation
// belge sur la facturation (AR du 21/12/2012).
montant = montant.setScale(2, RoundingMode.HALF_UP);
```

CC-COM.2 — Pas de code laissé en commentaire

Du code commenté pose une question sans réponse à tous ceux qui le liront : est-ce important ? Est-ce que quelqu'un compte le remettre ? Personne n'osera le supprimer, et il restera là des années.

À ÉVITER
```
public double calculerTarif() {
    // ancienne formule, a garder au cas ou
    // return prixBase * 1.21;
    return prixBase * tauxTva;
}
```

PRÉFÉREZ
```
public double calculerTarif() {
    return prixBase * tauxTva;
}
```

Supprimez. Si vous travaillez avec Git, la version précédente est conservée et retrouvable ; si vous n'utilisez pas encore Git, c'est une raison de plus de vous y mettre.

CC-COM.3 — Une Javadoc ne répète pas la signature

Une Javadoc qui paraphrase les noms des paramètres n'apporte rien et donne l'illusion que la méthode est documentée.

À ÉVITER
```
/**
 * Retire un montant.
 *
 * @param montant le montant
 * @return un boolean
 */
public boolean retirerArgent(double montant) { ... }
```

PRÉFÉREZ
```
/**
 * Retire un montant du compte, si le solde le permet.
 *
 * @param montant le montant a retirer, strictement positif
 * @return true si le retrait a eu lieu, false si le solde etait insuffisant
 */
public boolean retirerArgent(double montant) { ... }
```

Une Javadoc utile répond à ce que la signature ne dit pas : quelles valeurs sont acceptables, ce qui se passe dans les cas limites, ce qui est levé et quand. Le reste est déjà écrit juste en dessous.

Objets et encapsulation

Encapsuler ne consiste pas à rendre les champs privés puis à ouvrir un accesseur pour chacun : cela revient au même en plus verbeux. Encapsuler, c'est décider de ce que l'objet accepte de faire, pas de ce qu'il accepte de montrer.

CC-OBJ.1 — Demandez à l'objet d'agir, ne lui demandez pas ses entrailles

Une chaîne d'appels qui traverse plusieurs objets révèle leur structure interne à qui ne devrait pas la connaître. Le jour où l'un d'eux change, tout le code qui traversait la chaîne casse.

À ÉVITER
```
partie.getVainqueur().getCompte().crediter(10);
```

PRÉFÉREZ
```
partie.crediterVainqueur(10);
```

L'appelant sait maintenant une seule chose : une partie peut créditer son vainqueur. Il ignore qu'un `Joueur` possède un `Compte`, et le jour où ce n'est plus le cas, seule la classe `Partie` est à modifier.

Cette règle porte le nom de loi de Déméter, résumée par la formule « ne parlez qu'à vos amis proches ». En pratique, méfiez-vous de tout enchaînement de plus de deux points sur des objets de votre domaine.

Attention à ne pas sur-appliquer. La règle vise les objets de votre domaine, pas les chaînes de construction ou les bibliothèques conçues ainsi. `LocalDate.now().getYear()` est parfaitement correct — vous l'avez d'ailleurs écrit en `POO.3`.

CC-OBJ.2 — Un accesseur n'est pas une obligation

Générer mécaniquement un getter et un setter pour chaque champ annule l'encapsulation qu'on venait de mettre en place. Chaque accesseur est une décision : ce champ doit-il vraiment être lisible, et surtout modifiable, depuis l'extérieur ?

À ÉVITER
```
public class Partie {
    private List<Joueur> joueurs;

    public List<Joueur> getJoueurs()              { return joueurs; }
    public void setJoueurs(List<Joueur> joueurs)  { this.joueurs = joueurs; }
}

// n'importe qui peut vider la partie de ses joueurs :
partie.getJoueurs().clear();
```

PRÉFÉREZ

```
public class Partie {
    private final List<Joueur> joueurs = new ArrayList<>();

    public void ajouterJoueur(Joueur joueur) {
        joueurs.add(joueur);
    }

    public int nombreDeJoueurs() {
        return joueurs.size();
    }
}
```

La seconde version n'expose aucune liste. Elle expose deux opérations, qui sont ce dont l'appelant a réellement besoin. Et le jour où une partie doit refuser un cinquième joueur, il y a un endroit où écrire cette règle.

Vous avez rencontré cette question dès POO.2, à propos du champ `age` : un getter sans setter, un setter, ou rien. La réponse était « rien », parce que la donnée elle-même n'avait pas lieu d'être.

Traiter les erreurs

Ces six règles s'appliquent au module EXC, mais elles décident aussi de la lisibilité du code qui les appelle. Mal traitée, une erreur oblige chaque appelant à vérifier, à commenter, et finalement à oublier de vérifier.

CC-ERR.1 — Une erreur se signale par une exception, pas par une valeur de retour

Une valeur de retour qui code un échec oblige l'appelant à s'en souvenir. Rien dans le langage ne l'y contraint, et le jour où il oublie, le programme continue avec des données fausses — sans rien signaler.

À ÉVITER

```
public boolean retirer(double montant) {
    if (solde < montant) {
        return false;
    }
    solde -= montant;
    return true;
}

// chez l'appelant, rien n'oblige a ecrire ceci :
compte.retirer(500);   // et si le retrait a echoue ?
```

PRÉFÉREZ

```
public void retirer(double montant) {
    if (solde < montant) {
        throw new SoldeInsuffisantException(
                "Retrait de " + montant + " refuse : solde de " + solde);
    }
    solde -= montant;
}

// chez l'appelant, l'echec ne peut plus passer inapercu :
compte.retirer(500);
```

Nuance. Un booléen de retour n'est pas toujours mauvais : il est bien meilleur qu'un `System.out.println` dans la classe (voir CC-MET.7). La question à se poser est : s'agit-il d'une *réponse* ou d'une *erreur* ? `estDecouvert()` retourne légitimement un booléen, parce que les deux réponses sont normales. `retirer()` ne le devrait pas, parce que l'échec est anormal.

CC-ERR.2 — Ne retournez pas null

Retourner `null` revient à demander à chaque appelant d'écrire une vérification, pour toujours. Un seul oubli et le programme s'arrête sur une `NullPointerException`, à un endroit qui n'a rien à voir avec la cause.

À ÉVITER

```
public Joueur chercherJoueur(String pseudo) {
```

```
    for (Joueur joueur : joueurs) {
        if (joueur.getPseudo().equals(pseudo)) {
            return joueur;
        }
    }
    return null;
}
```

PRÉFÉREZ

```
public Joueur chercherJoueur(String pseudo) {
    for (Joueur joueur : joueurs) {
        if (joueur.getPseudo().equals(pseudo)) {
            return joueur;
        }
    }
    throw new JoueurInconnuException("Aucun joueur nomme " + pseudo);
}
```

La règle vaut aussi pour les collections, et là c'est encore plus simple : retournez une collection vide, jamais `null`. L'appelant peut alors faire son `for` sans rien vérifier, et la boucle ne tourne simplement pas.

À ÉVITER

```
if (partie.getJoueurs() != null) {
    for (Joueur joueur : partie.getJoueurs()) { ... }
}
```

PRÉFÉREZ

```
for (Joueur joueur : partie.getJoueurs()) { ... }
```

CC-ERR.3 — Ne passez pas null en paramètre

Le symétrique de la règle précédente. Une méthode qui accepte `null` pour l'un de ses paramètres doit le tester, et son comportement devient difficile à décrire. Quand un paramètre est facultatif, écrivez plutôt deux méthodes.

À ÉVITER

```
public void demarrer(Joueur premier, Joueur second, Strategie strategie) {
    if (strategie == null) {
        strategie = new StrategieTimide();
    }
    ...
}

partie.demarrer(alice, bob, null);
```

PRÉFÉREZ

```
public void demarrer(Joueur premier, Joueur second) {
    demarrer(premier, second, new StrategieTimide());
}

public void demarrer(Joueur premier, Joueur second, Strategie strategie) {
    ...
}

partie.demarrer(alice, bob);
```

La surcharge dit explicitement ce que veut l'appelant, et la méthode principale n'a plus de cas particulier à traiter.

CC-ERR.4 — Le message d'exception dit ce qui a échoué et avec quelle valeur

Le message d'une exception est souvent la seule information dont vous disposerez pour comprendre un incident. Un message qui ne nomme ni l'opération ni la valeur fautive ne sert à rien.

À ÉVITER

```
throw new IllegalArgumentException("Erreur");
throw new IllegalArgumentException("Valeur invalide");
throw new IllegalArgumentException();
```

PRÉFÉREZ

```
throw new IllegalArgumentException(
        "Nombre de cartes invalide : " + nombre + " (attendu entre 1 et 52)");
```

Trois ingrédients suffisent : ce qui était attendu, ce qui a été reçu, et le nom de ce qu'on manipulait. Écrivez le message en pensant à quelqu'un qui lira la trace sans avoir le code sous les yeux.

CC-ERR.5 — Définissez vos exceptions selon ce que l'appelant doit distinguer

Quand vous créez une classe d'exception, la question n'est pas « quelle erreur technique s'est produite ». Elle est : celui qui l'attrape va-t-il réagir différemment ? Si la réponse est non, une seule classe suffit.

À ÉVITER
```
try {
    compte.retirer(montant);
} catch (SoldeInsuffisantException e) {
    afficherErreur(e.getMessage());
} catch (CompteBloqueException e) {
    afficherErreur(e.getMessage());
} catch (MontantNegatifException e) {
    afficherErreur(e.getMessage());
}
```

Trois classes d'exception, trois blocs identiques : la distinction n'a servi à rien, sinon à alourdir chaque appelant. Une seule classe, avec un message précis, aurait suffi.

PRÉFÉREZ
```
try {
    compte.retirer(montant);
} catch (RetraitImpossibleException e) {
    afficherErreur(e.getMessage());
}
```

L'inverse est vrai aussi : si l'appelant doit vraiment réagir autrement — proposer un découvert dans un cas, appeler le service client dans l'autre — alors deux classes distinctes se justifient pleinement. C'est le besoin de l'appelant qui décide, jamais la cause technique.

> **Ce que le message ne doit pas faire**
>
> Ne construisez pas le message avec des informations sensibles — mot de passe, numéro de compte complet. La trace d'exception finit souvent dans un fichier de log que beaucoup de gens peuvent lire.
>
> N'écrivez pas non plus le message en pensant à l'utilisateur final : ce n'est pas lui qui le lira. Le message d'exception est destiné au développeur ; le message affiché à l'écran est une autre affaire, décidée par l'appelant.

CC-ERR.6 — N'avalez pas une exception en silence

Un `catch` vide fait taire l'erreur sans la résoudre. Le programme continue comme si tout allait bien, dans un état faux, et l'incident ressurgit plus loin, à un endroit qui n'a plus rien à voir avec sa cause. Le plantage était bruyant mais honnête ; ce silence ment.

À ÉVITER
```
try {
    journal.enregistrer(partie);
} catch (IOException e) {
    // ne devrait pas arriver
}
```

Trois réactions sont honnêtes. Traiter : redemander, proposer autre chose, prendre une valeur de repli que l'on assume. Informer : dire ce qui a échoué. Ou ne pas attraper du tout, et laisser l'exception remonter jusqu'à quelqu'un qui saura quoi en faire.

PRÉFÉREZ
```
public void terminer(Partie partie) throws IOException {
    journal.enregistrer(partie);    // c'est l'appelant qui decidera
}
```

Si vous ne savez pas quoi faire d'une exception à cet endroit, ce n'est pas l'endroit pour l'attraper : déclarez-la.

Quand l'objet exception ne sert plus

Attraper une exception pour redemander une saisie est un vrai traitement, même quand l'objet exception n'est plus utilisé. Nommez alors la variable ignoree : le nom dit que c'est voulu, et non oublié.

CC-ERR.7 — Une comparaison se protège de null

Appeler une méthode sur une variable qui vaut `null` lève une `NullPointerException` : il n'y a pas d'objet à qui demander. La comparaison est l'endroit où l'on se fait le plus souvent prendre.

À ÉVITER
```
if (titre.equals("Nana")) {   // plante si titre vaut null
    ...
}
```

PRÉFÉREZ
```
if (titre != null && titre.equals("Nana")) { ... }

if ("Nana".equals(titre)) { ... }          // la constante d'abord

if (Objects.equals(titre, "Nana")) { ... } // java.util.Objects
```

Dans la première écriture, l'ordre est essentiel : l'opérateur `&&` s'arrête dès que la première condition est fausse, et la seconde n'est jamais évaluée. Inverser les deux tests remet le plantage. La deuxième met la constante devant : elle n'est jamais `null`, et l'appel retourne simplement `false`. La troisième gère le `null` des deux côtés.

Nuance. Ces tests sont un filet de sécurité. La vraie protection est en amont : ne pas produire de `null`, ni en retour (CC-ERR.2), ni en paramètre (CC-ERR.3). Un code qui teste la nullité partout signale souvent un `null` qui n'aurait pas dû exister.

Écrire des tests

Une classe de test est du code que vous relirez et modifierez pendant toute la vie du projet. Elle mérite donc le même soin que le reste — et elle obéit à quelques règles qui lui sont propres.

CC-TST.1 — Le code de test se soigne comme le code de production

Un test illisible ne sera pas maintenu ; un test non maintenu finit désactivé ; une suite désactivée ne protège plus rien. Les règles de nommage et de longueur s'appliquent donc intégralement aux tests.

À ÉVITER
```
@Test
void test1() {
    Compte c = new Compte();
    c.deposer(100);
    c.retirerArgent(30);
    assertEquals(70, c.getSolde());
}
```

PRÉFÉREZ
```
@Test
@DisplayName("Un retrait diminue le solde du montant retire")
void unRetraitDiminueLeSolde() {
    Compte compte = new Compte();
    compte.deposer(100);

    compte.retirerArgent(30);

    assertEquals(70, compte.getSolde());
}
```

Le nom de la méthode de test énonce le comportement attendu. Quand le test passe au rouge, ce nom est la première chose que vous lirez — et souvent la seule dont vous aurez besoin pour comprendre ce qui est cassé.

Notez aussi la disposition en trois temps, séparés par une ligne vide : préparer, agir, vérifier. Elle se retrouve dans presque tous les tests et se repère d'un coup d'œil.

CC-TST.2 — Un test vérifie une seule chose

Quand un test vérifie trois comportements, son échec ne vous dit pas lequel est cassé, et les vérifications situées après la première qui échoue ne sont même pas exécutées.

À ÉVITER

```
@Test
void testCompte() {
    Compte compte = new Compte();
    compte.deposer(100);
    assertEquals(100, compte.getSolde());

    assertTrue(compte.retirerArgent(30));
    assertEquals(70, compte.getSolde());

    assertFalse(compte.retirerArgent(1000));
    assertEquals(70, compte.getSolde());
}
```

PRÉFÉREZ

```
@Test
void unDepotAugmenteLeSolde() { ... }

@Test
void unRetraitPossibleDiminueLeSolde() { ... }

@Test
void unRetraitTropGrandEstRefuseEtLaisseLeSoldeIntact() { ... }
```

Plusieurs assertions dans un même test restent acceptables tant qu'elles décrivent un seul comportement — vérifier le résultat retourné et l'état de l'objet après la même action, par exemple. La règle porte sur le nombre de comportements, pas sur le nombre de lignes `assertXxx`.

> **Un test sans assertion ne teste rien**
>
> Une méthode annotée `@Test` qui se contente d'appeler du code passe au vert quoi qu'il arrive : le seul cas où elle échoue est celui d'une exception inattendue. Elle donne donc une impression de couverture sans vérifier aucun comportement.
>
> C'est l'erreur la plus fréquente dans les classes de test rendues à l'examen. Avant de considérer un test comme écrit, cherchez son assertion.

CC-TST.3 — Un test est rapide, indépendant, répétable et auto-vérifiant

Ces qualités sont résumées par l'acronyme F.I.R.S.T., cinq mots anglais.

Qualité	
`Fast`	Rapide. Une suite lente n'est plus lancée, donc ne sert plus à rien.
`Independent`	Indépendant. Aucun test ne suppose qu'un autre a tourné avant lui, ni dans quel ordre.
`Repeatable`	Répétable. Même résultat sur votre machine, celle du voisin, et dans six mois.
`Self-validating`	Auto-vérifiant. Le test dit vert ou rouge, sans que personne n'ait à lire une sortie console.
`Timely`	Écrit tôt. Idéalement avant le code, comme dans l'exercice `JUN.1`.

La répétabilité est celle qui vous piégera le plus tôt. Un test qui s'appuie sur la date du jour, sur un tirage aléatoire ou sur un fichier présent seulement chez vous passera au rouge un matin, sans que rien n'ait changé dans le code.

À ÉVITER

```
@Test
void ageDeLaPersonne() {
    Personne personne = new Personne();
```

```
    personne.setAnneeDeNaissance(1990);

    assertEquals(36, personne.getAge());     // faux des le 1er janvier
}
```

PRÉFÉREZ

```
@Test
void ageDeLaPersonne() {
    Personne personne = new Personne();
    personne.setAnneeDeNaissance(1990);

    assertEquals(35, personne.computeAge(2025));
}
```

C'est exactement la raison pour laquelle `computeAge(int annee)` existe à côté de `getAge()` en `POO.3` : le noyau déterministe est testable, la commodité qui dépend du jour ne l'est qu'indirectement.

CC-TST.4 — Un test se termine par une assertion

Sans assertion, un test appelle du code et ne vérifie rien : il passe au vert quoi qu'il arrive, sauf si une exception l'interrompt. Il donne l'illusion d'une vérification, ce qui est pire que pas de test du tout. C'est aussi le défaut le plus fréquent dans les copies d'examen.

À ÉVITER

```
@Test
void reserverDeuxPlaces() throws SalleCompleteException {
    Salle salle = new Salle(100);

    salle.reserver(2);

    System.out.println(salle.placesLibres());   // affiche, ne verifie rien
}
```

PRÉFÉREZ

```
@Test
void reserverDeuxPlaces() throws SalleCompleteException {
    Salle salle = new Salle(100);

    salle.reserver(2);

    assertEquals(98, salle.placesLibres(),
            "Deux places reservees sur 100 doivent en laisser 98");
}
```

Un test qui vérifie qu'une exception est levée a lui aussi son assertion : `assertThrows`. Pour vous assurer qu'un test peut échouer, faites-le échouer une fois : changez l'attendu, et regardez-le passer au rouge. Un test qu'on n'a jamais vu rouge ne prouve rien.

Structurer une classe

Les deux dernières règles sont les mêmes que celles sur les méthodes, appliquées un cran au-dessus. Ce n'est pas un hasard : une bonne conception répète le même principe à chaque échelle.

CC-CLS.1 — Une classe n'a qu'une raison de changer

Décrivez la classe en une phrase. Si vous devez employer « et », elle porte plusieurs responsabilités — et chacune la fera évoluer pour des motifs différents, à des moments différents, souvent par des personnes différentes.

À ÉVITER

```
public class Facture {
    public double calculerTotal()   { ... }   // regles metier
    public void enregistrerEnBase() { ... }   // persistance
    public String enHtml()          { ... }   // presentation
}
```

PRÉFÉREZ

```
public class Facture {
    public double calculerTotal() { ... }
}

public class DepotFactures {
    public void enregistrer(Facture facture) { ... }
}

public class FactureEnHtml {
    public String convertir(Facture facture) { ... }
}
```

Trois raisons de changer, trois classes : un taux de TVA qui bouge, une base de données qu'on remplace, une maquette qu'on refait. Chacune touche un seul fichier.

Vous reconnaîtrez ici la même faute qu'en CC-MET.7 avec le `CompteBancaire` qui affichait ses messages : mélanger le traitement et la présentation. C'est le cas de figure le plus courant, et le plus coûteux.

CC-CLS.2 — Une classe reste petite

La taille d'une classe ne se mesure pas en lignes mais en responsabilités. Une classe de quarante méthodes est presque toujours plusieurs classes qui s'ignorent.

Un indice fiable : regardez quels champs chaque méthode utilise. Si un groupe de méthodes ne touche qu'à un sous-ensemble des champs, et un autre groupe au reste, vous tenez deux classes.

À ÉVITER

```
public class Joueur {
    private String pseudo;
    private String email;
    private String rue;
    private String codePostal;
    private String ville;
    private String pays;

    public String adresseFormatee()   { ... }   // n'utilise que les 4 derniers
    public boolean adresseValide()    { ... }   // idem
    public boolean emailValide()      { ... }
}
```

PRÉFÉREZ

```
public class Joueur {
    private String pseudo;
    private String email;
    private Adresse adresse;

    public boolean emailValide() { ... }
}

public record Adresse(String rue, String codePostal, String ville, String pays) {
    public String formatee() { ... }
    public boolean valide()  { ... }
}
```

C'est le même regroupement qu'en CC-MET.6 : des données qui voyagent ensemble et que les mêmes méthodes manipulent forment une chose, et cette chose mérite un nom.

CC-CLS.3 — Ne vous répétez pas

Toute duplication est une occasion de divergence. Le jour où la règle change, on modifie une copie et on oublie les autres — et le programme se met à donner deux réponses différentes à la même question.

La duplication la plus fréquente, et la plus coûteuse, se cache entre les sous-classes d'une même hiérarchie :

À ÉVITER

```
public class JeuDeCartes extends Jeu {
    public double calculerTarif() {
        double tarif = prixBase * duree;
```

```
        if (nombreDeJoueurs > 4) {
            tarif *= 1.2;
        }
        return tarif;
    }
}

public class JeuDePlateau extends Jeu {
    public double calculerTarif() {
        double tarif = prixBase * duree;
        if (nombreDeJoueurs > 4) {
            tarif *= 1.2;
        }
        return tarif * 0.9;
    }
}
```

Les deux méthodes sont identiques à un coefficient près. Remontez ce qui est commun dans la classe abstraite et ne laissez descendre que ce qui varie :

PRÉFÉREZ

```
public abstract class Jeu {

    private static final int    SEUIL_GRAND_GROUPE     = 4;
    private static final double MAJORATION_GRAND_GROUPE = 1.2;

    public double calculerTarif() {
        double tarif = prixBase * duree;
        if (nombreDeJoueurs > SEUIL_GRAND_GROUPE) {
            tarif *= MAJORATION_GRAND_GROUPE;
        }
        return tarif * coefficient();
    }

    protected abstract double coefficient();
}

public class JeuDePlateau extends Jeu {
    protected double coefficient() {
        return 0.9;
    }
}
```

Le calcul du tarif n'existe plus qu'à un seul endroit, et chaque sous-classe se réduit à ce qui la distingue vraiment. Notez que les constantes de CC-NOM.7 sont venues d'elles-mêmes : dès qu'on rassemble, les nombres magiques deviennent visibles.

C'est une erreur très fréquente à l'examen. Devant une hiérarchie, le réflexe est de recopier la méthode dans chaque sous-classe et d'y changer un détail. Le bon réflexe est de chercher ce qui est commun d'abord.

CC-CLS.4 — Préférez le polymorphisme à une chaîne de tests sur un type

Quand une méthode demande à un objet de quel type il est pour décider quoi faire, c'est que le travail devrait être confié à l'objet lui-même.

À ÉVITER

```
public double calculerTarif(Jeu jeu) {
    if (jeu instanceof JeuDeCartes) {
        return jeu.getPrixBase() * 1.0;
    } else if (jeu instanceof JeuDePlateau) {
        return jeu.getPrixBase() * 0.9;
    } else if (jeu instanceof JeuVideo) {
        return jeu.getPrixBase() * 1.5;
    }
    throw new IllegalArgumentException("Type de jeu inconnu");
}
```

PRÉFÉREZ

```
// chaque sous-classe repond pour elle-meme
```

```
double tarif = jeu.calculerTarif();
```

La différence décisive apparaît le jour où vous ajoutez un type de jeu. Avec la chaîne de tests, il faut retrouver tous les endroits qui testent le type — et rien ne vous signale ceux que vous avez oubliés : le programme compile et se contente de lever son exception à l'exécution.

Avec le polymorphisme, la méthode `abstract` de la classe mère oblige le compilateur à réclamer l'implémentation manquante. L'oubli devient impossible.

Une exception à la règle. Un `switch` sur une énumération d'états ou de commandes est parfaitement légitime — ce n'est pas un test de type. La règle vise les cascades de `instanceof` et les `switch` sur un champ qui code le type d'un objet.

La règle du boy-scout

Trente-quatre règles, cela peut décourager. Elles ne se prennent pas d'un bloc, et personne n'écrit du code parfait du premier coup — pas même l'auteur du livre dont elles viennent.

Les scouts américains ont une consigne de camp : laissez l'emplacement plus propre que vous ne l'avez trouvé. Appliquée au code, elle donne une règle praticable : chaque fois que vous ouvrez un fichier, améliorez une chose. Un nom plus clair, un commentaire remplacé par une méthode nommée, une constante à la place d'un nombre magique.

Une seule chose, pas dix. C'est suffisant pour qu'un programme s'améliore avec le temps au lieu de se dégrader — ce qui est, en pratique, le seul objectif atteignable.

Pour aller plus loin

Clean Code: A Handbook of Agile Software Craftsmanship, Robert C. Martin, Prentice Hall, 2008. Les chapitres 2 (noms), 3 (fonctions), 4 (commentaires), 6 (objets), 7 (erreurs), 9 (tests) et 10 (classes) correspondent aux sections de ce mémento.

Le site cleancoders.com propose les vidéos de l'auteur. La conférence « Clean Code » d'Uncle Bob, disponible en ligne, en donne un aperçu d'une heure.

En français, Julien Lucas présente ces principes dans une vidéo : https://www.youtube.com/watch?v=VioMQpZMtnA.

Index

A

À quoi sert Maven (TEC-MVN.1) 44
Accesseur n'est pas une obligation (CC-OBJ.2) 60
Aller à la déclaration, et revenir (IDE.23) 26
Avertissements et leurs corrections (IDE.15) 20

B

Booléen se retourne tel quel (CC-MET.9) 58

C

Cannot resolve symbol (ERR.2) 31
Ce qu'est un gestionnaire de versions (TEC-GIT.1) 47
Ce que contient vraiment une variable (TEC-EXE.3) 43
Chercher n'importe quoi (IDE.24) 26
classe abstraite 5
Classe n'a qu'une raison de changer (CC-CLS.1) 66
Classe reste petite (CC-CLS.2) 67
Classpath (TEC-CMP.3) 42
Code de test se soigne comme le code de production (CC-TST.1) 64
collection 5
Commentaire nécessaire signale un code à réécrire (CC-COM.1) 59
Comparaison se protège de null (CC-ERR.7) 64
Complétion et import automatique (IDE.11) 18
Cycle de vie (TEC-MVN.4) 45

D

Débogueur (IDE.19) 23
Déclarez une variable près de l'endroit où elle sert (CC-NOM.8) 54
Définissez vos exceptions selon ce que l'appelant doit distinguer (CC-ERR.5) 63
Demandez à l'objet d'agir, ne lui demandez pas ses entrailles (CC-OBJ.1) 60
Déplacer une classe (IDE.21) 25
Dépôt du fil rouge, et comment le parcourir (TEC-GIT.3) 47
Dépôt local (TEC-MVN.5) 46
Deux classes du même nom (ERR.5) 32
Deux paramètres au plus (CC-MET.6) 57
Dossier target (TEC-CMP.4) 42

E

énumération 5
Erreur se signale par une exception, pas par une valeur de retour (CC-ERR.1) 61
événement 6
exception 5
Exercice inachevé bloque tous les autres (ERR.9) 34
Extraire une archive au bon endroit (TEC-ZIP.1) 46
Extraire une méthode (IDE.22) 26

F

Fenêtre Maven (IDE.9) 17
Fenêtre Project, créer une classe (IDE.8) 16
Fichier disparaît de la fenêtre Project (ERR.10) 34

G

GameHub (fil rouge) 7, 9
Générer accesseurs et constructeurs (IDE.12) 19

H

héritage 5

I

Installer IntelliJ (IDE.2) 12
Installer Scene Builder (IDE.29) 30
Installer un JDK depuis IntelliJ (IDE.5) 13
interface 5

J

Javadoc 9
Javadoc ne répète pas la signature (CC-COM.3) 59
JavaFX 5, 9
JavaFX
rien à configurer, ou presque (IDE.28) 28
JavaFX runtime components are missing (ERR.7) 33
JDK isn't specified for module (ERR.1) 31
JDK ne vient pas avec IntelliJ (IDE.3) 12
JVM 9
JVM, JRE, JDK
qui fait quoi (TEC-PLA.1) 37

L

lambda 5
Lancer les tests et lire le rapport (IDE.17) 21
Lancer malgré une erreur de compilation (ERR.6) 32
Lancer une classe (IDE.16) 21
Lire la Javadoc sans quitter l'éditeur (IDE.14) 19
Longueur du nom suit la portée (CC-NOM.6) 53

M

main 8
Message d'exception dit ce qui a échoué et avec quelle valeur (CC-ERR.4) 62
Méthode agit ou répond, jamais les deux (CC-MET.8) 58
Méthode ne fait qu'une seule chose (CC-MET.3) 56
Méthode reste courte (CC-MET.2) 55
Modèles
psvm, sout, fori (IDE.13) 19
Modules Maven ont disparu
plus rien ne se lance (ERR.11) 35

N

N'avalez pas une exception en silence (CC-ERR.6) 63
Ne pas écraser son propre travail (TEC-ZIP.2) 47
Ne passez pas null en paramètre (CC-ERR.3) 62
Ne retournez pas null (CC-ERR.2) 61
Ne vous répétez pas (CC-CLS.3) 67
Nom communique l'intention (CC-NOM.1) 51
Nom de la méthode dit ce qu'elle fait (CC-MET.1) 55
Nom n'encode pas le type (CC-NOM.4) 52
Nom ne dit pas autre chose que ce qu'il désigne (CC-NOM.2) 51
Nom se prononce (CC-NOM.3) 52
Noms pour les choses, verbes pour les actions, prédicats pour les booléens (CC-NOM.5) 52

O

Ouvrir et fermer les fenêtres d'outils (IDE.10) 18
Ouvrir par le pom.xml, jamais par le dossier (IDE.4) 12

P

package 7
Pas d'effet de bord (CC-MET.7) 57
Pas de code laissé en commentaire (CC-COM.2) 59
Pas de drapeau booléen en paramètre (CC-MET.5) 56
Pas de flèche verte sur une classe de test (ERR.4) 32
Pas de nombre magique nommez-le (CC-NOM.7) 53
Passer des arguments au programme (IDE.18) 23
Pile (TEC-EXE.1) 43
Plugins et apparence (IDE.27) 27
Pom.xml (TEC-MVN.2) 44
Pourquoi certains dossiers sont ignorés (TEC-GIT.2) 47
Pourquoi Java tourne partout (TEC-CMP.2) 42
Préférez le polymorphisme à une chaîne de tests sur un type (CC-CLS.4) 68
Projet, plusieurs modules Maven (TEC-MVN.3) 45

Q

Quatre zones (IDE.7) 15
Que contient ma machine (TEC-PLA.4) 38
Quelle édition d'IntelliJ (IDE.1) 12

R

Raccourcis à connaître (IDE.26) 27
Ramasse-miettes (TEC-EXE.4) 44
record 5
référence 4, 8
Renommer (IDE.20) 25

S

Source level should be in ... 21 (ERR.8) 33
Structure d'un fichier (IDE.25) 26

T

tableau 5, 7
Tas (TEC-EXE.2) 43
test (vérifier son code) 6, 7, 8
Test est rapide, indépendant, répétable et auto-vérifiant (CC-TST.3) 65
Test se termine par une assertion (CC-TST.4) 66
test unitaire 5, 7
Test vérifie une seule chose (CC-TST.2) 65
Tout est rouge sans raison (ERR.3) 32
Trois éditions (TEC-PLA.2) 38
Trois états de votre programme (TEC-CMP.1) 41
type générique 5

U

UnsupportedOperationException 7

V

Vérifier le SDK du projet (IDE.6) 14
Versions et le support long terme (TEC-PLA.3) 38
Vocabulaire est cohérent d'une méthode à l'autre (CC-MET.4) 56